Dossier de modélisation — Jeu d'échecs en ligne

TIC315 — Modélisation objet avec UML, Licence 3. Projet 4 (jeu de plateau en ligne), variante échecs.

1. Introduction et périmètre 1

2. Besoins : cas d'utilisation 2

3. Modèle structurel 6

4. Modèle comportemental 8

5. Contraintes OCL 11

6. Conception 14

7. Architecture 23

8. Traçabilité 24

Annexe A — Cahier des charges 26

Annexe B — Diagrammes complémentaires 29

Annexe C — Contraintes OCL complètes 42

1. Introduction et périmètre

Ce dossier présente la modélisation d'un site de jeu d'échecs en ligne. Un joueur crée un compte, se connecte, puis joue contre un autre joueur ou contre l'ordinateur, avec une pendule pour les parties entre joueurs. Il consulte ses classements et revoit ses parties terminées. Un administrateur gère les comptes.

Acteur	Type	Rôle
Joueur	Principal	Crée son compte, joue, consulte ses classements et ses parties
Administrateur	Principal	Consulte, suspend et réactive les comptes
Service d'authentification	Secondaire, externe	Vérifie l'identité du joueur à la connexion
Moteur d'échecs (Stockfish)	Secondaire, externe	Calcule les coups de l'ordinateur à la force demandée

Périmètre. Le modèle décrit les règles officielles complètes des échecs, la pendule, le classement Elo par famille de cadence (bullet, blitz, rapide) et l'historique des parties. Sont hors périmètre : les spectateurs, la discussion entre joueurs, les tournois, les parties en différé, l'analyse d'une partie par le moteur et l'application mobile native. Le cahier des charges complet, avec ses besoins (BF) et ses règles de gestion (RG), est en annexe A.

Démarche. Le dossier suit les jalons du cours : les besoins, le modèle structurel, le modèle comportemental, les contraintes OCL, la conception avec ses patrons, puis l'architecture. Une matrice de traçabilité relie ces étapes.

2. Besoins : cas d'utilisation

Le système compte 13 cas d'utilisation. Les trois cas détaillés ci-dessous sont les plus importants et les plus risqués : UC8 porte les règles du jeu, UC6 le déroulement d'une partie avec pendule, UC7 la dépendance à un système externe.

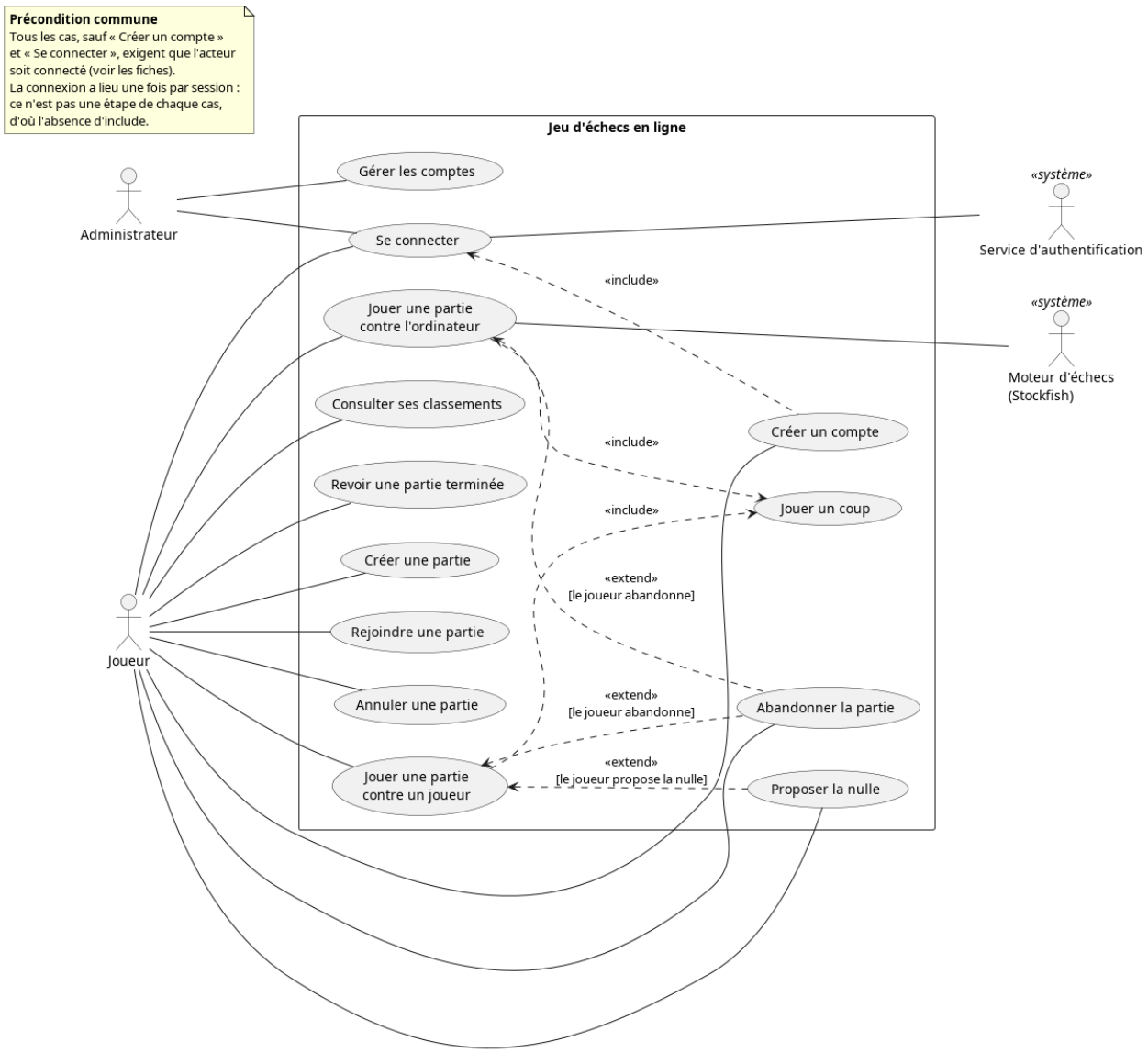


Figure 1 — Diagramme de cas d'utilisation

UC8 — Jouer un coup

Objectif. Le joueur déplace une de ses pièces conformément aux règles ; le système enregistre le coup et donne le trait à l'adversaire.

Acteurs. Principal : Joueur. Secondaire : aucun. Ce cas est inclus dans UC6 et UC7 ; le coup de l'ordinateur est décrit dans UC7.

Préconditions.

- Le joueur est connecté.
- Une partie à laquelle il participe est en cours.
- C'est au joueur d'avoir le trait (RG3).

Déclencheur. Le joueur sélectionne une pièce de sa couleur.

Scénario nominal.

1. Le joueur sélectionne une de ses pièces.
2. Le système affiche les cases où cette pièce peut légalement aller.
3. Le joueur choisit une case de destination.
4. Le système vérifie que le coup est légal (RG1, RG2).
5. Le système exécute le coup : il déplace la pièce et retire la pièce adverse prise, s'il y en a une.
6. Le système enregistre le coup dans l'historique de la partie (RG7).
7. Dans une partie avec pendule, le système arrête la pendule du joueur, lui ajoute l'incrément éventuel et démarre celle de l'adversaire (RG5, RG10).
8. Le système vérifie que le coup ne termine pas la partie (RG4, RG9).
9. Le système donne le trait à l'adversaire et affiche le nouveau plateau aux deux camps.

Scénarios alternatifs.

- **1a. La pièce n'a aucun coup légal.** Le système le signale ; retour à l'étape 1.
- **3a. Le joueur change d'avis.** Il sélectionne une autre de ses pièces ; retour à l'étape 2.
- **4a. Le coup est illégal.** Le système refuse le coup et en indique le motif (par exemple : « votre roi serait en échec ») ; le plateau est inchangé ; retour à l'étape 1.
- **5a. Promotion.** Le pion atteint la dernière rangée : le système demande la pièce choisie (dame, tour, fou ou cavalier), le joueur choisit, le système remplace le pion ; reprise à l'étape 6.
- **5b. Roque.** Le joueur déplace son roi de deux cases vers une tour : le système déplace aussi la tour de l'autre côté du roi ; reprise à l'étape 6.
- **5c. Prise en passant.** Le système retire le pion adverse qui vient d'avancer de deux cases à côté du pion du joueur ; reprise à l'étape 6.
- **7a. Une proposition de nulle de l'adversaire attend une réponse.** Le coup joué vaut refus : le système retire la proposition (RG4) ; reprise à l'étape 8.
- **8a. Échec et mat.** La partie se termine par la victoire du joueur ; le cas s'achève et le cas englobant (UC6 ou UC7) enregistre le résultat.
- **8b. Pat ou nulle automatique** (triple répétition, 50 coups, matériel insuffisant). La partie se termine par une nulle ; le cas s'achève et le cas englobant enregistre le résultat.
- **8c. Échec sans mat.** Le système signale l'échec au camp adverse ; reprise à l'étape 9.

Exceptions.

- **E1. Le temps du joueur s'écoule avant qu'il ait joué.** Aucun coup n'est enregistré ; la fin de partie au temps est traitée par UC6 (RG5).
- **E2. Le joueur perd la connexion pendant son tour.** Sa pendule continue de tourner (RG5).

Postconditions.

- Succès : le coup figure dans l'historique, le plateau est à jour et le trait est à l'adversaire, ou la partie est terminée.
- Échec : le plateau et l'historique sont inchangés ; le trait reste au joueur.

Règles de gestion. RG1, RG2, RG3, RG4, RG5, RG7, RG9, RG10.

UC6 — Jouer une partie contre un joueur

Objectif. Deux joueurs s'affrontent jusqu'à un résultat (victoire ou nulle), qui est enregistré et pris en compte dans leur classement.

Acteurs. Principal : Joueur ; les deux adversaires jouent ce même rôle. Secondaire : aucun.

Préconditions.

- Les deux joueurs sont connectés et leurs comptes ne sont pas suspendus (RG8).
- Une partie créée (UC3) vient d'être rejointe (UC4).
- Aucun des deux joueurs n'a d'autre partie en attente ou en cours (RG11).

Déclencheur. Le second joueur rejoint la partie, ou accepte l'invitation.

Scénario nominal.

1. Le système attribue les couleurs selon le choix du créateur, au hasard si celui-ci l'a demandé.
2. Le système place les pièces en position initiale et passe la partie à l'état « en cours ».
3. Le système règle les deux pendules sur la cadence choisie et démarre celle des blancs (RG5).
4. Le joueur qui a le trait joue un coup (inclusion de UC8 — Jouer un coup).
5. L'étape 4 se répète, chaque joueur à son tour, jusqu'à ce qu'un coup termine la partie : mat, pat ou nulle automatique (RG4, RG9).
6. Le système arrête les pendules et enregistre le résultat.
7. Le système met à jour le classement des deux joueurs dans la famille de la cadence de la partie (RG6).
8. Le système affiche le résultat aux deux joueurs ; la partie rejoint leur historique (RG7).

Scénarios alternatifs.

- **4a. Annulation en début de partie.** Tant que chaque camp n'a pas joué son premier coup, un joueur annule la partie : elle est annulée, sans résultat ni effet sur le classement (RG16) ; fin du cas.
- **5a. Temps écoulé.** La pendule d'un joueur atteint zéro : le système termine la partie par sa défaite, ou par une nulle si son adversaire n'a pas le matériel pour mater (RG5) ; reprise à l'étape 6.
- **5b. Proposition de nulle** (extension UC9). Un joueur propose la nulle : si l'adversaire accepte, la partie se termine par une nulle et reprend à l'étape 6 ; s'il refuse, ou s'il joue un coup sans répondre, ce qui vaut refus (RG4), la partie continue à l'étape 4.
- **5c. Abandon** (extension UC10). Un joueur abandonne : la partie se termine par sa défaite ; reprise à l'étape 6.
- **5d. Suspension.** L'administrateur suspend le compte d'un des joueurs : la partie est considérée comme abandonnée par lui (RG8) ; reprise à l'étape 6.

Exceptions.

- **E1. Un joueur perd la connexion.** Sa pendule continue de tourner ; s'il ne revient pas avant la fin de son temps, l'alternatif 5a s'applique. La pendule garantit qu'une partie ne peut pas rester bloquée.

Postconditions.

- La partie est terminée ; son résultat et tous ses coups sont enregistrés dans l'ordre. Dans le cas 4a, elle est annulée, sans résultat.
- Le classement des deux joueurs est mis à jour, sauf si la partie est annulée.
- Aucun des deux joueurs n'a plus de partie en cours.

Règles de gestion. RG3, RG4, RG5, RG6, RG7, RG8, RG9, RG11, RG16.

UC7 — Jouer une partie contre l'ordinateur

Objectif. Le joueur affronte l'ordinateur à la force de son choix, pour s'entraîner, sans effet sur son classement.

Acteurs. Principal : Joueur. Secondaire : Moteur d'échecs (Stockfish).

Préconditions.

- Le joueur est connecté et son compte n'est pas suspendu (RG8).
- Le joueur n'a pas d'autre partie en attente ou en cours (RG11).

Déclencheur. Le joueur demande une partie contre l'ordinateur.

Scénario nominal.

1. Le joueur demande une partie contre l'ordinateur.
2. Le système propose les forces de jeu disponibles et le choix de la couleur.
3. Le joueur choisit une force et sa couleur.
4. Le système crée la partie sans pendule (RG10), place les pièces et la passe à l'état « en cours ».
5. Le joueur, qui a les blancs, joue un coup (inclusion de UC8 — Jouer un coup).
6. Le système transmet la position au moteur d'échecs, avec la force choisie.
7. Le moteur d'échecs renvoie son coup.
8. Le système vérifie que ce coup est légal, l'exécute et l'enregistre dans l'historique (RG1, RG7).
9. Les étapes 5 à 8 se répètent jusqu'à ce qu'un coup termine la partie : mat, pat ou nulle automatique (RG4, RG9).
10. Le système enregistre le résultat, sans modifier le classement (RG6), et l'affiche au joueur.

Scénarios alternatifs.

- **3a. Le joueur choisit les noirs.** L'ordinateur joue le premier coup : le déroulement commence à l'étape 6.
- **5a. Abandon** (extension UC10). Le joueur abandonne : la partie se termine par sa défaite ; reprise à l'étape 10.
- **5b. Annulation en début de partie.** Tant que chaque camp n'a pas joué son premier coup, le joueur annule la partie : elle est annulée, sans résultat (RG16) ; fin du cas.
- **8a. Le coup du moteur termine la partie.** Mat, pat ou nulle automatique : reprise à l'étape 10.

Exceptions.

- **E1. Le moteur ne répond pas dans le délai prévu.** Le système renouvelle la demande une fois ; en cas de nouvel échec, il informe le joueur et la partie est annulée : elle n'a pas de résultat (RG15).
- **E2. Le moteur renvoie un coup illégal.** Le système rejette ce coup et redemande un coup au moteur.

Postconditions.

- Succès : la partie est terminée ; son résultat et tous ses coups sont enregistrés dans l'ordre.
- Dans tous les cas, le classement du joueur est inchangé.

Règles de gestion. RG1, RG2, RG4, RG6, RG7, RG9, RG10, RG11, RG16.

3. Modèle structurel

Le modèle du domaine décrit les concepts du jeu, sans aucune classe technique ; les énumérations sont détaillées en annexe B. Le diagramme d'objets valide ce modèle sur une situation réelle : la position après 1. e4 d5 2. exd5, en vue partielle.

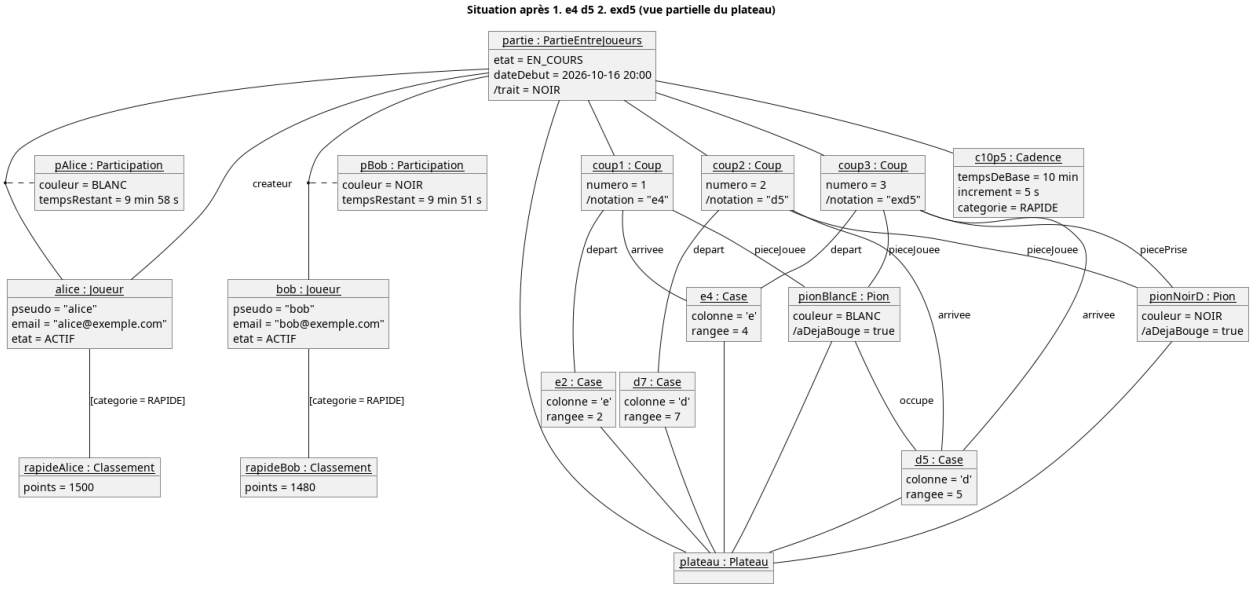


Figure 2 — Diagramme d'objets de validation

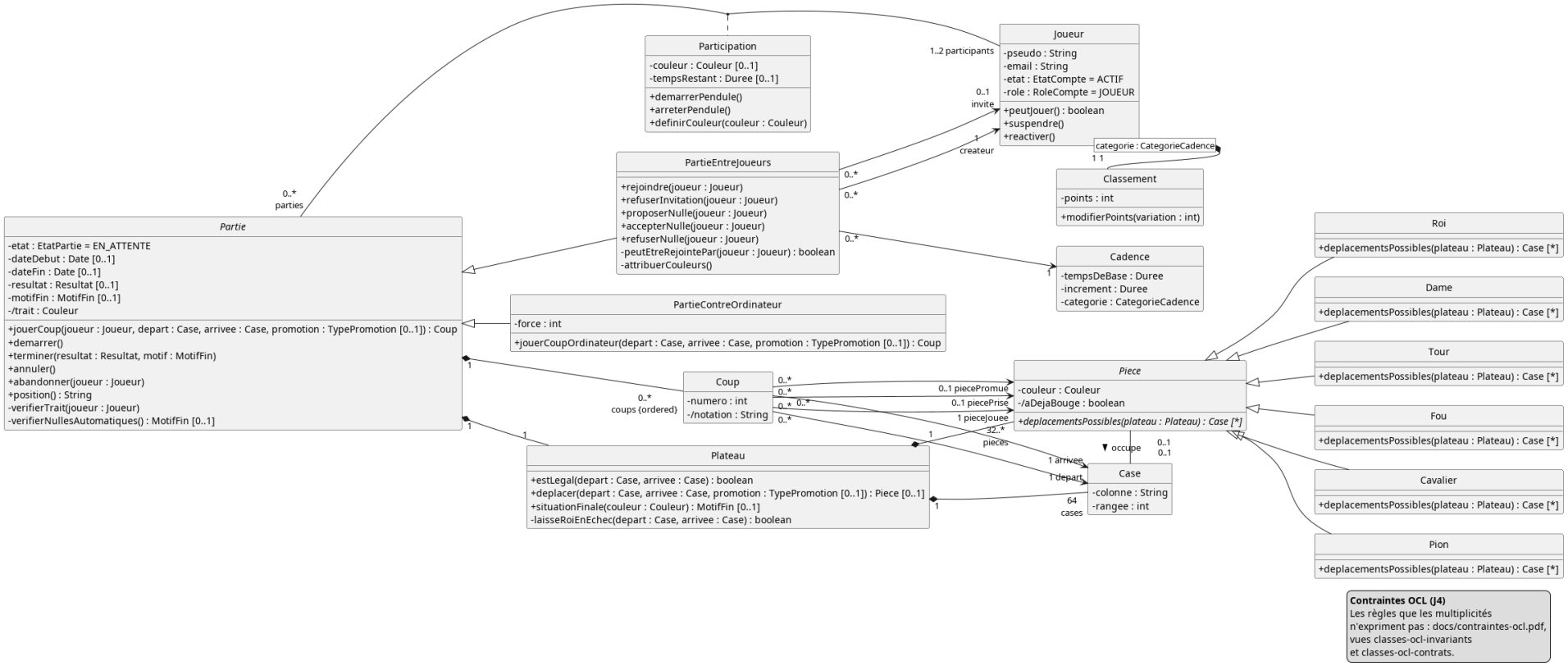


Figure 3 — Diagramme de classes du domaine

4. Modèle comportemental

Les trois séquences couvrent les cas principaux. Le diagramme d'états suit la classe Partie, avec le tour et la proposition de nulle en régions parallèles ; le diagramme d'activité suit une partie entre joueurs, de sa création à son début.

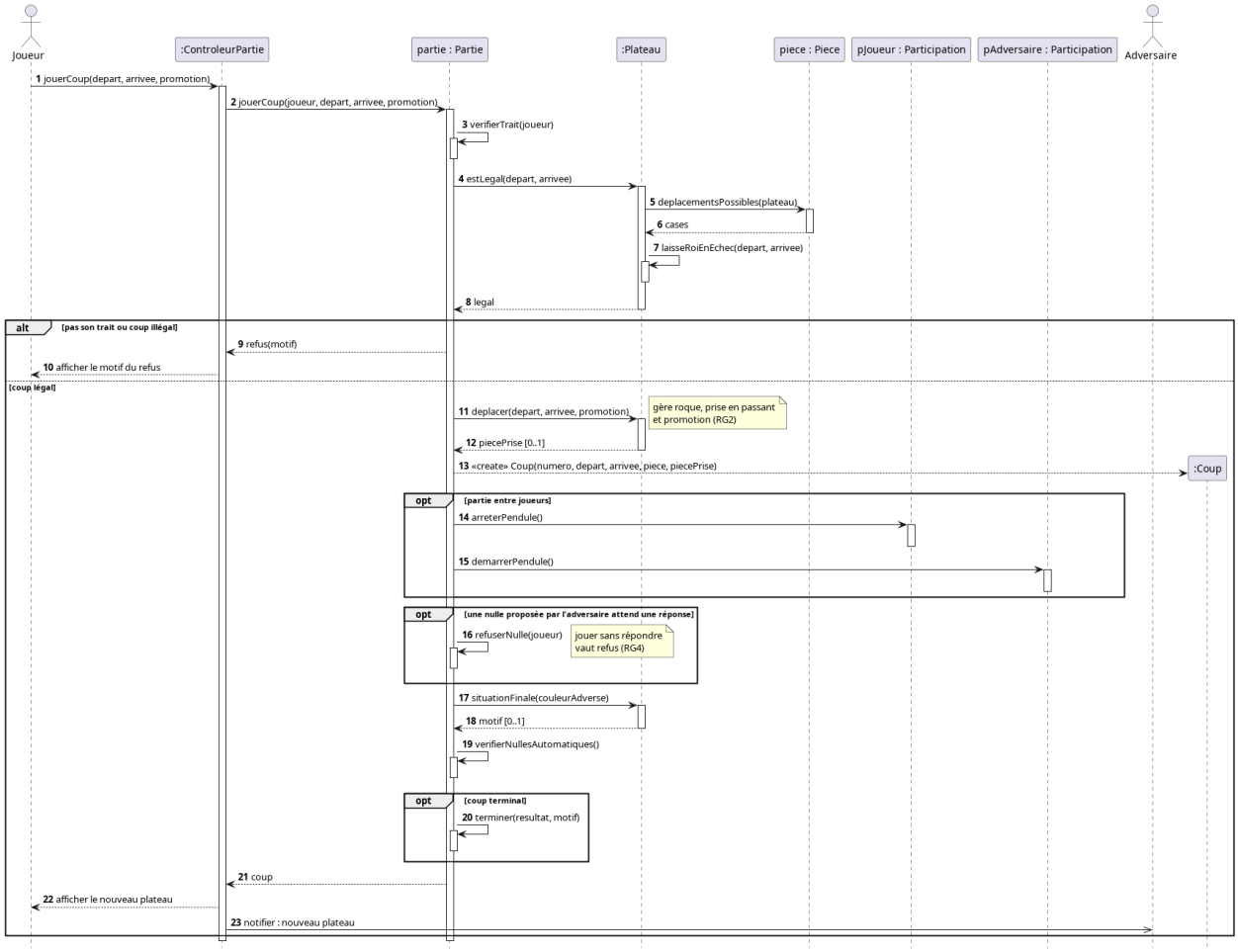


Figure 4 — Séquence : jouer un coup

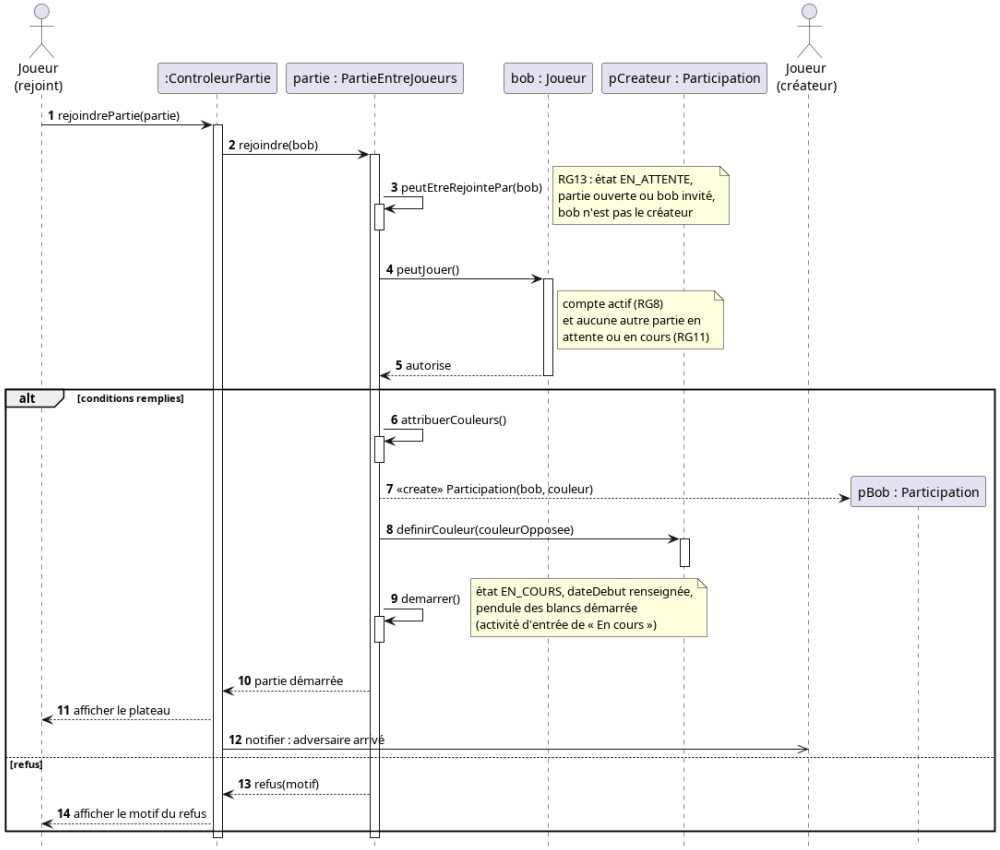


Figure 5 — Séquence : rejoindre une partie

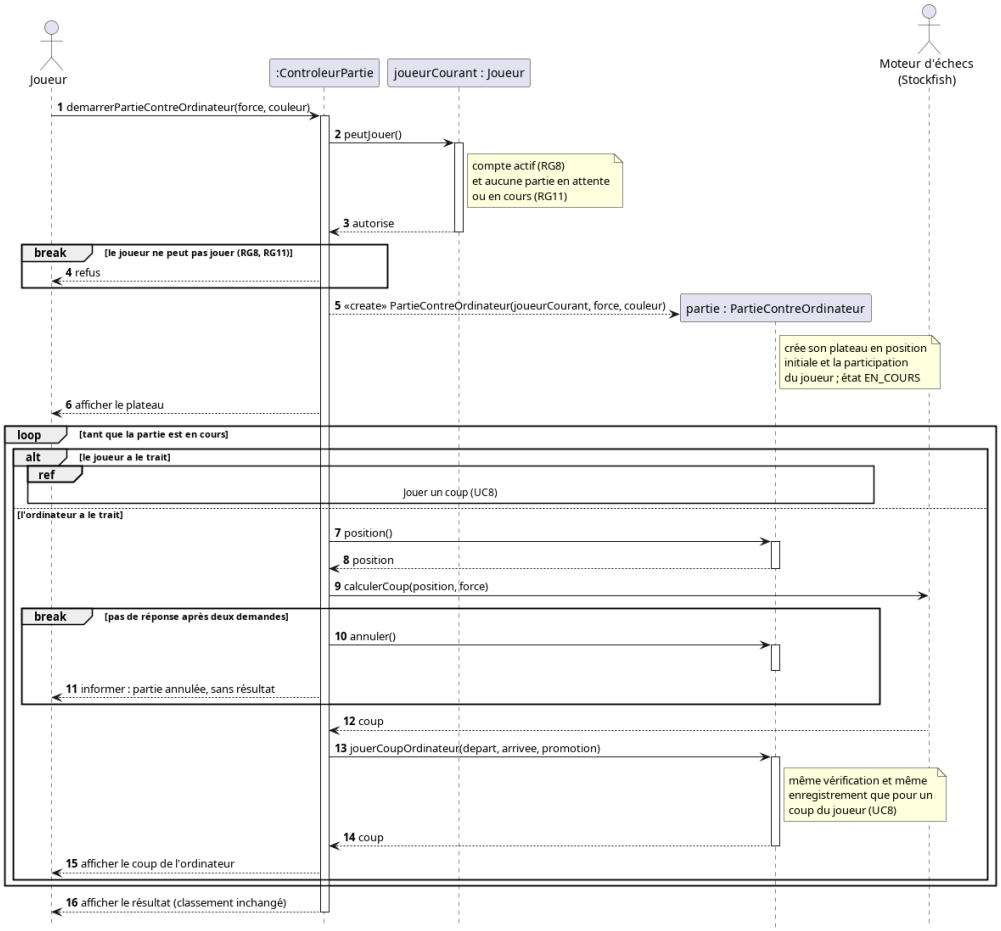


Figure 6 — Séquence : jouer contre l'ordinateur

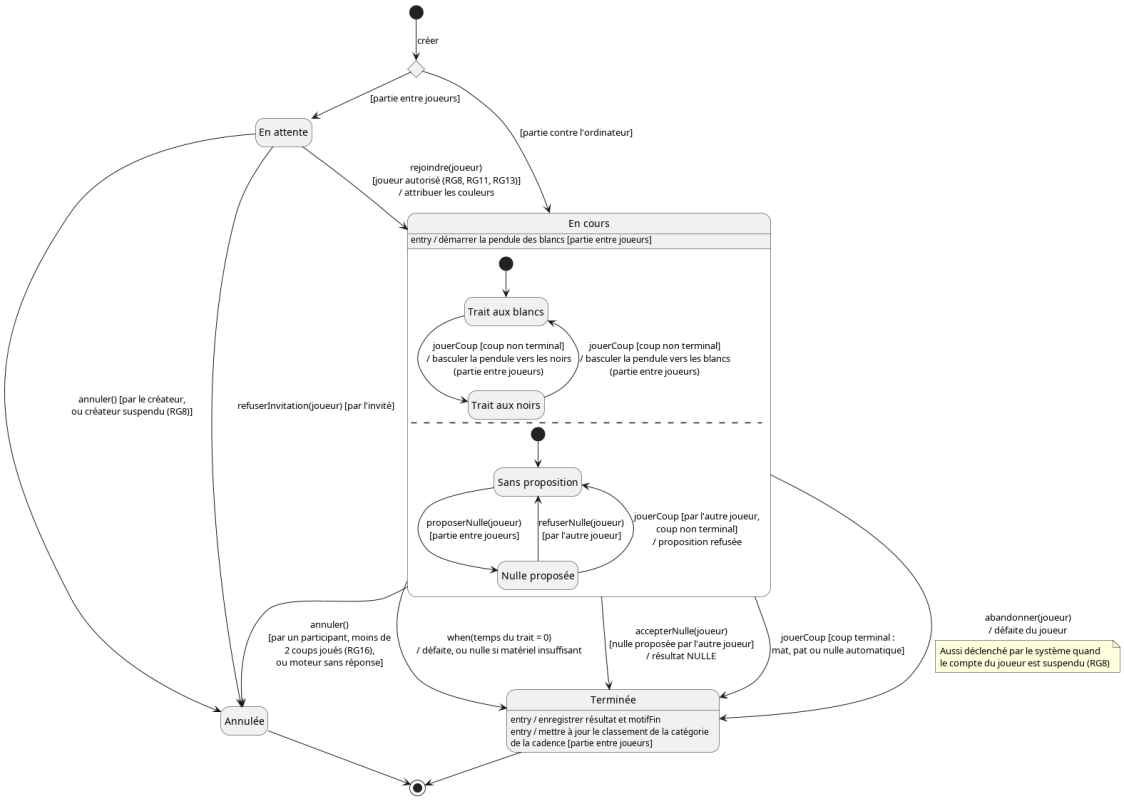


Figure 7 — Diagramme d'états de la partie

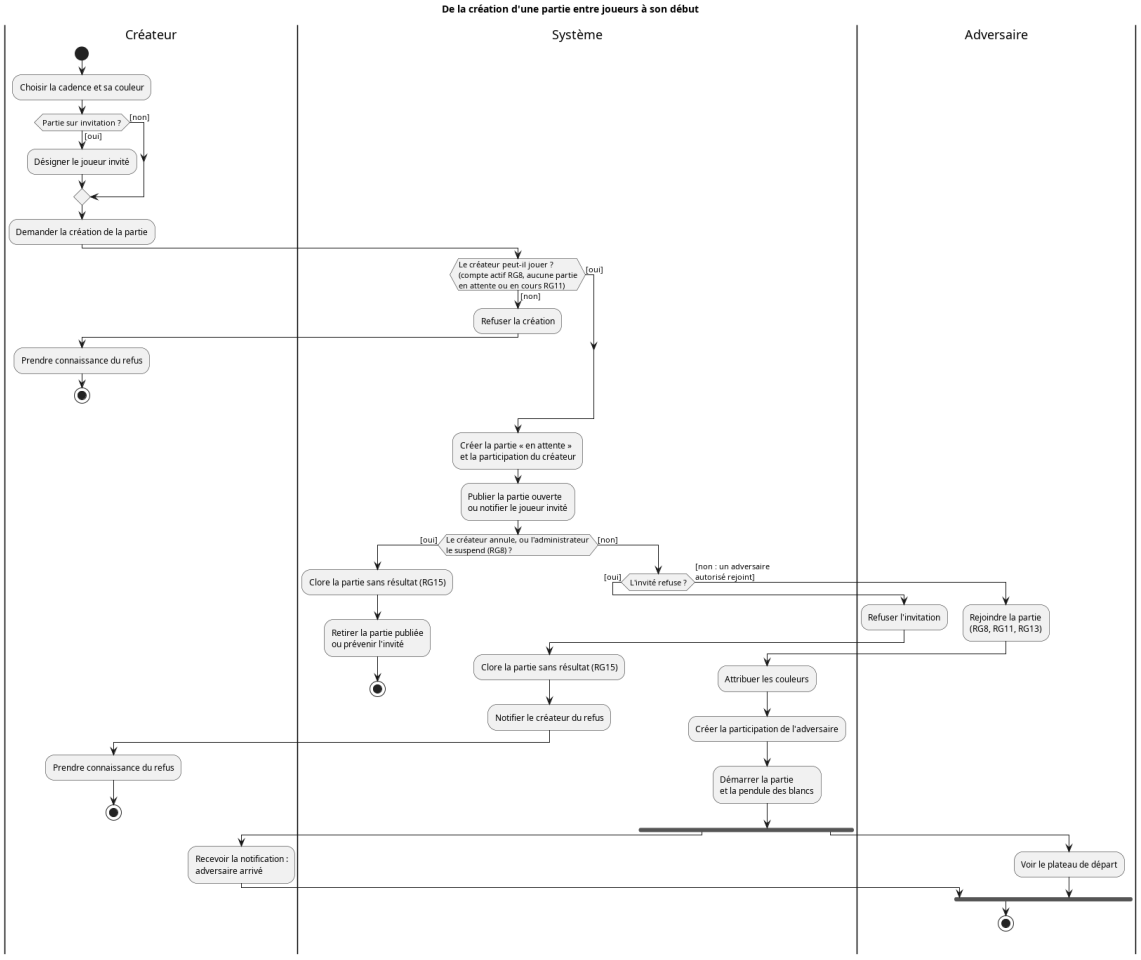


Figure 8 — Diagramme d'activité : démarrer une partie

5. Contraintes OCL

Ces contraintes expriment les règles que les multiplicités ne savent pas dire. Ce chapitre en présente huit groupes, qui couvrent toutes les sortes de contraintes. Les quinze sont détaillées en annexe C, et les vues du modèle qui les portent sont en annexe B.

5.1 Conventions et hypothèses

- **Navigation.** On suit les noms de rôle du diagramme : `participants`, `parties`, `createur`, `invite`, `coups`, `depart`, `arrivee`, `pieceJouee`, `piecePrise`, `piecePromue`, `cases`, `pieces`. Une extrémité sans nom de rôle se désigne par le nom de sa classe en minuscules : `plateau`, `partie`, `cadence`, `classement`.
- **Classe-association.** Depuis une partie ou un joueur, `participation` donne les objets Participation. Depuis une Participation, chaque extrémité s'atteint par son nom de rôle : `p.participants` est le joueur, `p.parties` la partie.
- **Association qualifiée.** `j.classement[c]` est le classement du joueur `j` dans la catégorie `c`.
- **Types.** OCL n'a pas de type Date ni Duree : comme le cours, on suppose ces deux types munis des comparaisons usuelles. Les attributs `int` et `boolean` du diagramme sont les types OCL `Integer` et `Boolean`.
- **Préconditions et refus.** Une opération appelée depuis l'interface reçoit des demandes du réseau : elle ne fait pas confiance à l'appelant et vérifie elle-même ses préconditions (`verifierTrait`, `peutEtreRejointePar`, `peutJouer`). Une précondition non remplie produit le refus montré dans les diagrammes de séquence ; la postcondition décrit le cas où l'opération aboutit.

5.2 Opération auxiliaire

Trois contraintes ont besoin de savoir si une partie est « active », c'est-à-dire en attente ou en cours. On le définit une fois.

```
context Partie
def: estActive() : Boolean =
  self.etat = EtatPartie::EN_ATTENTE or self.etat = EtatPartie::EN_COURS
```

5.3 Joueurs

C2 — Une partie à la fois (RG11)

Un joueur participe à au plus une partie en attente ou en cours.

```
context Joueur
inv unePartieALaFois: self.parties->select(p | p.estActive())->size() <= 1
```

La multiplicité `0..*` côté `parties` est nécessaire pour l'historique (RG7) : seule une contrainte peut limiter les parties actives sans limiter les parties terminées.

5.4 Plateau

C3 — Les 64 cases de l'échiquier (aucune RG : règle du jeu)

Chaque case a une colonne de a à h et une rangée de 1 à 8, et deux cases d'un même plateau n'ont pas les mêmes coordonnées.

```
context Case
inv coordonneesValides:
  Set{'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'}->includes(self.colonne)
  and self.rangee >= 1 and self.rangee <= 8

context Plateau
inv casesDistinctes:
```

```
  self.cases->forAll(c1, c2 |
    c1 <> c2 implies (c1.colonne <> c2.colonne or c1.rangee <> c2.rangee))
```

La multiplicité donne 64 cases ; ces deux invariants garantissent que ce sont les 64 cases de l'échiquier, chacune une seule fois.

5.5 Parties

C5 — Partie entre joueurs : créateur, invité et deux camps (RG3, RG13)

Le créateur participe toujours à sa partie ; il ne peut pas s'inviter lui-même ; tant que la partie attend, il est seul ; une fois commencée, elle oppose deux joueurs de couleurs différentes, dont l'invité s'il y en a un.

```
context PartieEntreJoueurs
inv createurParticipe: self.participants->includes(self.createur)
inv pasSaPropreInvitation:
  not self.invite.oclIsUndefined() implies self.invite <> self.createur
inv seulPendantLAttente:
  self.etat = EtatPartie::EN_ATTENTE implies self.participants = Set{self.createur}
inv deuxCampsUneFoisCommencee:
  (self.etat = EtatPartie::EN_COURS or self.etat = EtatPartie::TERMINEE) implies
    self.participants->size() = 2
    and self.participation->forAll(p | not p.couleur.oclIsUndefined())
    and self.participation->isUnique(p | p.couleur)
inv adversaireInvite:
  (not self.invite.oclIsUndefined() and self.participants->size() = 2)
    implies self.participants->includes(self.invite)
```

Le diagramme d'objets « impossible » (cas C) montre une partie que les multiplicités acceptent, avec deux participants blancs : `deuxCampsUneFoisCommencee` la refuse.

C6 — Résultat et motif, seulement pour une partie terminée (RG4, RG15)

Une partie a un résultat et un motif de fin si et seulement si elle est terminée ; une partie annulée n'en a donc pas. Le résultat doit être cohérent avec le motif.

```
context Partie
inv resultatSiTerminee:
  (self.etat = EtatPartie::TERMINEE) = not self.resultat.oclIsUndefined()
inv motifSiTerminee:
  (self.etat = EtatPartie::TERMINEE) = not self.motifFin.oclIsUndefined()
inv resultatCoherentAvecMotif:
  self.etat = EtatPartie::TERMINEE implies
    let motifsDeNulle : Set(MotifFin) =
      Set{MotifFin::PAT, MotifFin::ACCORD, MotifFin::TRIPLE_REPETITION,
          MotifFin::CINQUANTE_COUPS, MotifFin::MATERIEL_INSUFFISANT}
    in (motifsDeNulle->includes(self.motifFin) implies self.resultat = Resultat::NULLE)
       and ((self.motifFin = MotifFin::MAT or self.motifFin = MotifFin::ABANDON)
            implies self.resultat <> Resultat::NULLE)
```

Les multiplicités `[0..1]` disent seulement que les valeurs sont facultatives, pas quand elles doivent être présentes. `TEMPS_ECOULE` n'est pas contraint : il donne une défaite, ou une nulle si l'adversaire ne peut pas mater (RG5).

5.6 Coups

C8 — Un coup respecte les camps et le plateau (RG1, RG2, RG3)

Les blancs jouent les coups impairs et les noirs les coups pairs ; une pièce part d'une case et arrive sur une autre, du plateau de sa partie ; on ne prend que des pièces adverses et jamais le roi ; seul un pion arrivé sur la dernière rangée est promu, en une pièce de sa couleur qui n'est ni un roi ni un pion.

```
context Coup
```

```
inv alternanceDesCamps:
  self.pieceJouee.couleur =
    (if self.numero.mod(2) = 1 then Couleur::BLANC else Couleur::NOIR endif)
inv casesDuPlateauDeLaPartie:
  self.depart <> self.arrivee
  and self.partie.plateau.cases->includes(self.depart)
  and self.partie.plateau.cases->includes(self.arrivee)
  and self.partie.plateau.pieces->includes(self.pieceJouee)
inv priseDUnePieceAdverse:
  not self.piecePrise.oclIsUndefined() implies
    self.piecePrise.couleur <> self.pieceJouee.couleur
    and not self.piecePrise.oclIsKindOf(Roi)
inv promotionDUnPion:
  not self.piecePromue.oclIsUndefined() implies
    self.pieceJouee.oclIsTypeOf(Pion)
    and self.arrivee.rangee =
      (if self.pieceJouee.couleur = Couleur::BLANC then 8 else 1 endif)
    and self.piecePromue.couleur = self.pieceJouee.couleur
    and not self.piecePromue.oclIsKindOf(Roi)
    and not self.piecePromue.oclIsKindOf(Pion)
```

Les associations disent qu'un coup a une case de départ, une case d'arrivée et des pièces ; elles ne disent pas lesquelles sont permises. La légalité complète d'un déplacement (trajectoire, échec au roi) reste calculée par `Plateau::estLegal`.

5.7 Attributs dérivés et requêtes

C11 — Un joueur peut-il jouer ? (RG8, RG11, RG17)

```
context Joueur::peutJouer() : Boolean
body: self.role = RoleCompte::JOUEUR and self.etat = EtatCompte::ACTIF
      and not self.parties->exists(p | p.estActive())
```

Les séquences « rejoindre une partie » et « contre l'ordinateur » appellent `peutJouer()` : cette définition fixe exactement ce qu'elle vérifie.

5.8 Contrats d'opérations

C12 — Jouer un coup (RG1, RG3, RG7)

```
context Partie::jouerCoup(joueur : Joueur, depart : Case, arrivee : Case,
                          promotion : TypePromotion) : Coup
pre partieEnCours:   self.etat = EtatPartie::EN_COURS
pre joueurParticipe: self.participants->includes(joueur)
pre aLeTrait:
  self.participation->any(p | p.participants = joueur).couleur = self.trait
pre coupLegal:       self.plateau.estLegal(depart, arrivee)
post unCoupDePlus:   self.coups->size() = self.coups@pre->size() + 1
post coupEnregistre:
  result = self.coups->last()
  and result.depart = depart and result.arrivee = arrivee
post traitPasse:
  self.etat = EtatPartie::EN_COURS implies self.trait <> self.trait@pre
```

C14 — Terminer une partie : seul le bon classement change (RG6)

À la fin d'une partie entre joueurs, seul le classement de la catégorie de sa cadence peut changer ; à la fin d'une partie contre l'ordinateur, aucun ne change.

```
context Partie::terminer(resultat : Resultat, motif : MotifFin)
pre enCours: self.etat = EtatPartie::EN_COURS
post terminee:
  self.etat = EtatPartie::TERMINEE
  and self.resultat = resultat and self.motifFin = motif
post seulLeClassementDeLaCadenceChange:
  self.participants->forAll(j |
```

```
Set{CategorieCadence::BULLET, CategorieCadence::BLITZ, CategorieCadence::RAPIDE}
  ->forAll(c |
    (self.oclIsTypeOf(PartieContreOrdinateur)
     or c <> self.oclAsType(PartieEntreJoueurs).cadence.categorie)
    implies j.classement[c].points = j.classement[c].points@pre))
```

La formule de calcul des points (patron Stratégie) n'est pas fixée ici : le contrat dit seulement quels classements elle a le droit de modifier.

5.9 Récapitulatif

N°	Sorte	Contexte	Règles
—	def	Partie::estActive()	RG11
C1	inv (2)	Joueur	RG12
C2	inv	Joueur	RG11
C3	inv (2)	Case, Plateau	Règle du jeu
C4	inv (3)	PartieContreOrdinateur	RG5, RG10
C5	inv (5)	PartieEntreJoueurs	RG3, RG13
C6	inv (3)	Partie	RG4, RG5, RG15
C7	inv (2)	Partie	RG7
C8	inv (4)	Coup	RG1, RG2, RG3
C9	derive	Partie::trait	RG3
C10	derive	Piece::aDejaBouge	RG2
C11	body	Joueur::peutJouer()	RG8, RG11, RG17
C12	pre (4), post (3)	Partie::jouerCoup()	RG1, RG3, RG7
C13	pre (4), post (2)	PartieEntreJoueurs::rejoindre()	RG3, RG8, RG11, RG13
C14	pre, post (2)	Partie::terminer()	RG6
C15	pre, post (3)	Joueur::suspendre()	RG8, RG14

Soit 15 contraintes pour 8 exigées. Elles utilisent six des sept mots-clés du cours : `def`, `inv`, `derive`, `body`, `pre` et `post` ; seul `init` est inutile, les valeurs initiales figurant déjà dans le diagramme (`etat = EN_ATTENTE`, `etat = ACTIF`).

6. Conception

6.1 Du modèle du domaine au modèle de conception

Le modèle du domaine décrit les concepts du jeu. La conception ajoute ce qu'il faut pour coder : des types Java, un sens de navigation pour chaque association, des visibilités, et quelques classes techniques. Chaque écart est voulu :

Dans le domaine	Dans la conception	Raison
Types `Date`, `Duree`	`Instant`, `Duration` ; l'heure vient d'une `Clock`	Types standard de Java ; une horloge remplaçable rend la pendule testable en JUnit

Dans le domaine	Dans la conception	Raison
Association Partie — Joueur dans les deux sens	Partie → Joueur seulement ; Joueur garde un indicateur `occupe`	Casser le cycle entre `metier.partie` et `metier.joueur` (section 6.2) ; `peutJouer()` n'a pas à parcourir l'historique
Classe-association Participation	Classe ordinaire : Partie compose ses participations, chacune désigne son joueur	Règle de traduction du cours (§6.2)
Association qualifiée Joueur [categorie] — Classement	`Map<CategorieCadence, Classement>`	Traduction directe d'un qualificatif
Énumération `EtatPartie` et attribut `etat`	Classes d'état (patron État) ; `etat()` renvoie toujours l'énumération	Le comportement change avec l'état (section 6.3.1) ; l'énumération reste le vocabulaire commun à l'OCL, au diagramme d'états et à l'affichage
`/trait`, `/aDejaBouge`	`trait()` calculé sur le nombre de coups ; `aDejaBouge()` calculé sur un compteur de déplacements	Un attribut dérivé devient une méthode calculée (§6.2) ; la pièce n'a pas à remonter jusqu'aux coups de la partie
`annuler()`	`annuler(joueur)` et `annulerParLeSysteme()`	Qui annule change les règles : le créateur ou l'invité (RG13), un participant en début de partie (RG16) ; le système quand le créateur est suspendu (RG8) ou que le moteur ne répond plus (RG15)
`jouerCoupOrdinateur()` et `jouerCoup()`	Même chemin dans l'état et dans `executerCoup()`, avec un `joueur` absent pour l'ordinateur	Un coup de l'ordinateur subit la même vérification et le même enregistrement (UC7), mais l'ordinateur n'est pas un `Joueur`
Pendule et proposition de nulle, dans la séquence de `jouerCoup`	`apresCoup()` et `demarrer()`, qui n'agissent que dans `PartieEntreJoueurs`	Partie ne connaît pas la pendule (RG10) : polymorphisme plutôt qu'un test sur la sorte de partie
`deplacementsPossibles(plateau)`	`deplacementsPossibles(plateau, depart)`	C'est la case qui connaît sa pièce ; la pièce reçoit donc sa case de départ

Dans le domaine	Dans la conception	Raison
Déplacements d'une pièce	`casesAttaquees()` en plus de `deplacementsPossibles()`	Le roque demande si une case est attaquée : cette question ne doit pas dépendre des coups de l'autre roi, sinon le calcul tourne sans fin
Plateau compose ses 32 pièces ou plus	Les pièces en jeu s'atteignent par les cases, les pièces prises par leur coup	Une liste de pièces en plus doublerait ce que les cases disent déjà
`position()` du plateau	`position(trait)`	La notation FEN contient le camp qui a le trait, et le plateau ne le connaît pas
Énumération `MotifFin`	Dans `metier.jeu`	`Plateau::situationFinale()` la renvoie ; ailleurs, elle créerait un cycle
Événement `when(temps du trait = 0)`	`verifierTemps()`, appelée avant chaque coup et par une tâche périodique (J5)	Un événement de changement n'est pas un appel : il faut quelqu'un pour le constater
`Joueur::suspendre()` avec les effets de RG8	`Joueur::suspendre()` change l'état du compte ; `ControleurCompte::suspendre()` retrouve la partie active du joueur et l'abandonne ou l'annule	Le joueur ne connaît plus ses parties ; la postcondition `plusDePartieActive` (C15) est garantie par le contrôleur

Les accesseurs simples ne sont pas dessinés : le cours ne les prévoit que « si nécessaire » (§6.2), et ils n'ajoutent aucune décision.

6.2 Packages

Package	Contenu	Dépend de
`ihm`	`Application`, `VuePartie`	`controle`, `infrastructure`, `metier.partie`, `metier.jeu`
`controle`	`ControleurPartie`, `ControleurCompte`, `FilesDeParties`, `ArchivagePartie` ; interfaces `IPartie`, `ICompte`, `MoteurEchecs`, `ServiceAuthentification`, `DepotParties`, `DepotJoueurs`	`metier`
`metier.partie`	`Partie` et ses sous-classes, `Participation`, `Cadence`, les états, `ObservateurPartie`, `EtatPartie`, `Resultat`	`metier.jeu`, `metier.joueur`
`metier.jeu`	`Plateau`, `Case`, `Piece` et ses sous-classes, `Coup`, `Couleur`, `TypePromotion`, `MotifFin`	aucun
`metier.joueur`	`Joueur`, `Classement`, `CalculClassement`, `CalculElo`, `EtatCompte`, `CategorieCadence`	aucun
`infrastructure`	`AdaptateurStockfish`, `AdaptateurOpenIdConnect`, `DepotPartiesSql`, `DepotJoueursSql`	`controle`, `metier`

Les règles du cours (§2.3.2) sont respectées :

- **Aucun cycle.** Le seul cycle possible, entre `partie` et `joueur`, est cassé par la navigabilité Partie → Joueur ; celui entre `partie` et `jeu`, par le déplacement de `MotifFin`.
- **Dépendre de ce qui est stable.** Le métier ne dépend ni de l'interface, ni de la base, ni des services externes.
- **Inversion des dépendances.** `controle` définit les interfaces dont il a besoin ; `infrastructure` les implémente. Remplacer Stockfish ou la base ne touche ni le métier ni les contrôleurs.
- **Pas de package fourre-tout.** Chaque classe est rangée selon son rôle.

6.3 Patrons de conception

Le cours le rappelle : « pas de patron pour le plaisir ». Chaque patron ci-dessous répond à un problème précis du projet.

6.3.1 État — le comportement de la partie dépend de son état

Problème. Neuf événements (rejoindre, refuser l'invitation, annuler, jouer un coup, abandonner, proposer, accepter ou refuser la nulle, temps écoulé) n'ont pas le même effet selon que la partie est en attente, en cours, terminée ou annulée. Coder cela dans Partie mettrait un `switch` sur l'état au début de chaque méthode, et chaque nouvel état obligerait à modifier toutes ces méthodes.

Solution. Une classe abstraite `EtatDePartie` déclare une opération par événement du diagramme d'états ; par défaut, chacune refuse en levant `TransitionInterditeException`. Chaque état concret (`EtatEnAttente`, `EtatEnCours`, `EtatTerminee`, `EtatAnnulee`) ne redéfinit que les événements qu'il accepte. Partie délègue l'événement à son état courant ; l'état fait le travail en appelant Partie, puis change l'état si la transition le demande (`changerEtat`).

Où. `classes-conception-etat`, `sequence-conception-jouer-coup` (messages 5 à 8), `sequence-conception-terminer-partie` (messages 1 à 3).

Conséquences. Le diagramme d'états se lit directement dans le code : une classe par état, une méthode par transition. `verifier.py` contrôle que chaque événement du diagramme d'états a son opération dans `EtatDePartie`. États terminaux : `EtatTerminee` et `EtatAnnulee` n'acceptent rien. Le prix : cinq classes de plus, et des opérations de Partie en visibilité package pour que les états puissent agir sur elle.

Principes. Ouvert/fermé (un nouvel état est une nouvelle classe), responsabilité unique (chaque état connaît ses seules transitions), polymorphisme (GRASP).

6.3.2 Observateur, avec MVC — prévenir les deux écrans

Problème. Quand un joueur joue, l'écran de son adversaire doit se mettre à jour, comme ceux des deux joueurs quand la partie démarre, se termine ou reçoit une proposition de nulle. La séquence du J3 se termine d'ailleurs par « notifier : nouveau plateau » sans dire qui le fait. Si Partie appelait directement la couche web, le métier dépendrait de l'interface.

Solution. Partie tient une liste d'`ObservateurPartie` (interface du métier) et les notifie après chaque changement. `VuePartie`, dans `ihm`, réalise cette interface ; chaque joueur connecté a la sienne, abonnée à sa partie. Ces observateurs sont donc les écrans des deux participants : le système ne gère pas de spectateurs. Modèle « tiré » : la vue reçoit la partie et relit ce qui l'intéresse, ce qui garde une seule opération `actualiser(partie)` au lieu d'une par événement.

MVC. La vue transmet les actions au contrôleur, qui les délègue au modèle ; le modèle prévient la vue par l'Observateur, sans la connaître (cours, §5.4.6).

Où. `classes-conception-partie`, `classes-conception-controle`, `sequence-conception-jouer-coup` (messages 9 à 12).

Conséquences. Faible couplage entre le métier et l'interface. Attention, comme le dit le cours, aux observateurs oubliés : la vue d'un joueur qui quitte la partie doit se désabonner (`desabonner`).

6.3.3 Adaptateur — parler à Stockfish et au service d'authentification

Problème. Stockfish s'utilise par le protocole UCI (des commandes texte sur l'entrée standard d'un processus) ; le service d'authentification, par OpenID Connect. Aucune de ces interfaces ne doit apparaître dans les contrôleurs.

Solution. `controle` définit les interfaces dont il a besoin, dans son vocabulaire : `MoteurEchecs::calculerCoup(position, force)` et `ServiceAuthentification::verifierIdentite(jeton)`. `AdaptateurStockfish` et `AdaptateurOpenIdConnect`, dans `infrastructure`, les traduisent dans le protocole réel.

Où. `classes-conception-controle`.

Conséquences. On peut remplacer Stockfish par un autre moteur UCI, ou le fournisseur d'identité, sans toucher au reste. Les tests du contrôleur utilisent un faux moteur. Un moteur muet se traduit par `MoteurIndisponibleException`, qui mène à l'annulation de la partie (RG15).

6.3.4 Stratégie — la formule du classement

Problème. Le sujet dit que le classement évolue « selon la formule choisie » : la formule est un point de variation annoncé. Si elle était codée dans Partie, en changer obligerait à modifier la classe la plus centrale du modèle.

Solution. `PartieEntreJoueurs` délègue le calcul à une `CalculClassement`, reçue à sa création. `CalculElo` est l'implémentation retenue (section 6.6). La partie contre l'ordinateur ne calcule rien : `mettreAJourClassements()` n'est redéfinie que dans `PartieEntreJoueurs` (polymorphisme plutôt qu'un test sur la sorte de partie).

Où. `classes-conception-joueur`, `sequence-conception-terminer-partie` (messages 5 à 11).

Conséquences. Changer de formule, par exemple pour Glicko, utilisé par lichess et chess.com, revient à ajouter une classe (ouvert/fermé). La formule se teste seule, sans partie.

Limite assumée. Une seule formule est implémentée aujourd'hui. Le cours conseille d'introduire un patron « quand la rigidité apparaît » : ce patron se justifie ici par une variation annoncée par le sujet (GRASP Protection des variations), pas par deux variantes déjà présentes. C'est le moins fort des quatre patrons ; les patrons État et Observateur suffisent à eux seuls à l'exigence de deux patrons justifiés.

6.3.5 Une file d'actions par partie — plusieurs joueurs en même temps

Problème. Le serveur traite chaque joueur en parallèle. Si le temps d'un joueur s'écoule au moment où l'autre joue, deux traitements modifient la même partie, et un coup peut être enregistré dans une partie déjà terminée.

Solution. `FilesDeParties` donne à chaque partie sa file d'actions, traitées une par une et dans l'ordre. Les parties ne se gênent pas entre elles. Stockfish réfléchit hors de la file : la partie reste libre pendant son calcul, et un joueur peut abandonner sans l'attendre.

Pourquoi pas un verrou par partie ? C'était plus court à écrire. La file a été préférée : elle rend l'ordre des actions explicite, et aucune action ne peut oublier de prendre le verrou.

Conséquence. Une action ne doit jamais appeler la file depuis la file, ni y faire un appel lent : elle s'attendrait elle-même ou bloquerait la partie.

6.3.6 Patrons écartés

Patron	Pourquoi il n'est pas retenu
Singleton (pour le moteur)	Il cacherait la dépendance au moteur et empêcherait de le remplacer dans les tests ; le moteur est passé au contrôleur
Fabrique (pour les pièces)	Les pièces ne sont créées qu'à deux endroits connus : la position initiale du plateau et la promotion. Aucune variation à cacher
Composite	Aucune hiérarchie tout-partie récursive dans le domaine
Décorateur	Un seul canal de notification (la vue web) ; rien à cumuler
Commande	Servirait à annuler un coup ; reprendre un coup est hors périmètre

6.4 Répartition des responsabilités (GRASP)

Patron GRASP	Application
Expert en information	Le plateau vérifie la légalité d'un coup (il connaît les cases) ; chaque pièce calcule ses déplacements ; la partie vérifie le trait et les nulles automatiques (elle connaît les coups) ; la participation gère sa pendule (elle connaît son temps)
Créateur	La partie crée ses coups, ses participations et son plateau : elle les contient (composition)
Contrôleur	`ControleurPartie` et `ControleurCompte` reçoivent les actions des joueurs et délèguent, sans faire le travail
Polymorphisme	Déplacements par sorte de pièce ; comportement par état ; pendule et classements selon la sorte de partie
Fabrication pure	Les dépôts (`DepotParties`, `DepotJoueurs`) gardent la persistance hors du métier
Indirection	Le contrôleur entre la vue et le métier ; les interfaces entre le contrôleur et les services externes
Protection des variations	`CalculClassement`, `MoteurEchecs`, `ServiceAuthentification`
Faible couplage, forte cohésion	Packages sans cycle ; `metier.jeu` ne connaît ni joueur ni partie

6.5 Principes SOLID

Principe	Respect dans la conception
Responsabilité unique	La partie gère le déroulement, le plateau les règles de déplacement, la participation la pendule, le calcul la formule, le dépôt la persistance
Ouvert/fermé	Un nouvel état, une nouvelle formule de classement ou un nouveau moteur s'ajoutent par une nouvelle classe
Substitution de Liskov	Toute sous-classe de Partie, de Piece ou d'EtatDePartie s'utilise là où la classe mère est attendue ; un état qui n'accepte pas un événement le refuse par la même exception que la classe mère
Ségrégation des interfaces	Interfaces courtes et ciblées : `ObservateurPartie` (une opération), `CalculClassement` (une), `MoteurEchecs` (une), `ServiceAuthentification` (une) ; un dépôt par sorte d'objet

Principe	Respect dans la conception
Inversion des dépendances	Les contrôleurs dépendent d'interfaces, implémentées par l'infrastructure ; le métier possède `ObservateurPartie`, réalisée par l'interface utilisateur

6.6 Formule du classement (RG6)

Formule retenue : **Elo**, la formule de la FIDE.

- Score attendu d'un joueur de `points` contre un adversaire de `pointsAdversaire` : `1 / (1 + 10^((pointsAdversaire − points) / 400))`.
- Variation : `arrondi(K × (score − scoreAttendu))`, avec un score de 1 pour une victoire, 0,5 pour une nulle et 0 pour une défaite.
- **K = 20**, le coefficient de la FIDE pour les parties rapides et blitz. Chaque joueur commence à **1500** points dans chaque famille de cadence.
- Seul le classement de la famille de la cadence de la partie change ; les parties contre l'ordinateur et les parties annulées ne changent rien (RG6, RG15).

Exemple, qui servira de test JUnit au J5 : un joueur à 1500 bat un joueur à 1480. Le gagnant attendait 0,53 : il gagne 9 points, et le perdant en perd 9. Sur une nulle, le mieux classé perd 1 point et l'autre en gagne 1.

L'arrondi peut rendre la somme des deux variations différente de zéro d'un point ; c'est accepté, comme sur la plupart des sites. **Glicko** a été écarté pour le code : il demande de suivre en plus la fiabilité de chaque classement, et le patron Stratégie permet de l'ajouter plus tard.

6.7 Des contraintes OCL et des refus au code (J5)

Les règles écrites en OCL deviennent des vérifications explicites (cours, §6.2) :

Dans le modèle	Dans le code Java
Invariants (C1 à C8)	Vérifiés dans les constructeurs et les opérations qui modifient l'objet ; l'unicité (C1) par le dépôt, qui voit tous les joueurs
Préconditions (C12, C13, C14, C15)	Exceptions au début de l'opération : `TransitionInterditeException` si l'état refuse l'événement, `CoupRefuseException` si ce n'est pas le trait du joueur ou si le coup est illégal
Postconditions	Assertions des tests JUnit
Attributs dérivés (C9, C10)	Méthodes calculées `trait()` et `aDejaBouge()`
Requête `peutJouer()` (C11)	Méthode de Joueur, sur son état et l'indicateur `occupe`

Les tests JUnit du J5 suivront ces règles : un test par règle de gestion, au moins pour RG1 à RG6, RG11, RG13 et RG16. Ce tableau prépare la matrice de traçabilité du dossier final (cas d'utilisation, puis classes, puis tests).

6.8 Architecture et déploiement (J5)

L'application est un seul programme Spring Boot (`echecs.jar`), qui contient le serveur web et les pages du site. Le site est uniquement en ligne : les joueurs l'ouvrent dans leur navigateur, en HTTPS.

- **Hébergeur.** Le programme et Stockfish, qu'il lance comme un processus séparé, tournent dans un même conteneur Docker. L'image contient tout ce qu'il faut : le déploiement ne dépend pas de la machine.
- **Base de données.** Les données sont dans une base PostgreSQL hébergée, séparée du conteneur : un conteneur peut être recréé à tout moment, et ce qu'il contient est alors perdu. Les comptes et les classements survivent ainsi aux redémarrages.
- **Authentification.** `AdaptateurOpenIdConnect` joint un fournisseur d'identité externe, Google, conformément au sujet. Le site ne voit donc jamais de mot de passe.
- **Rôle.** Un compte a le rôle joueur ou administrateur (RG17). Le contrôleur vérifie ce rôle : la règle tient même si une page du site oublie de masquer un bouton. Le premier administrateur est créé au démarrage, à partir de la configuration.

Diagrammes de conception

Les classes du patron État, celles de `metier.jeu`, de `metier.joueur` et des couches `ihm`, `controle` et `infrastructure`, ainsi que les séquences de conception, sont en annexe B.

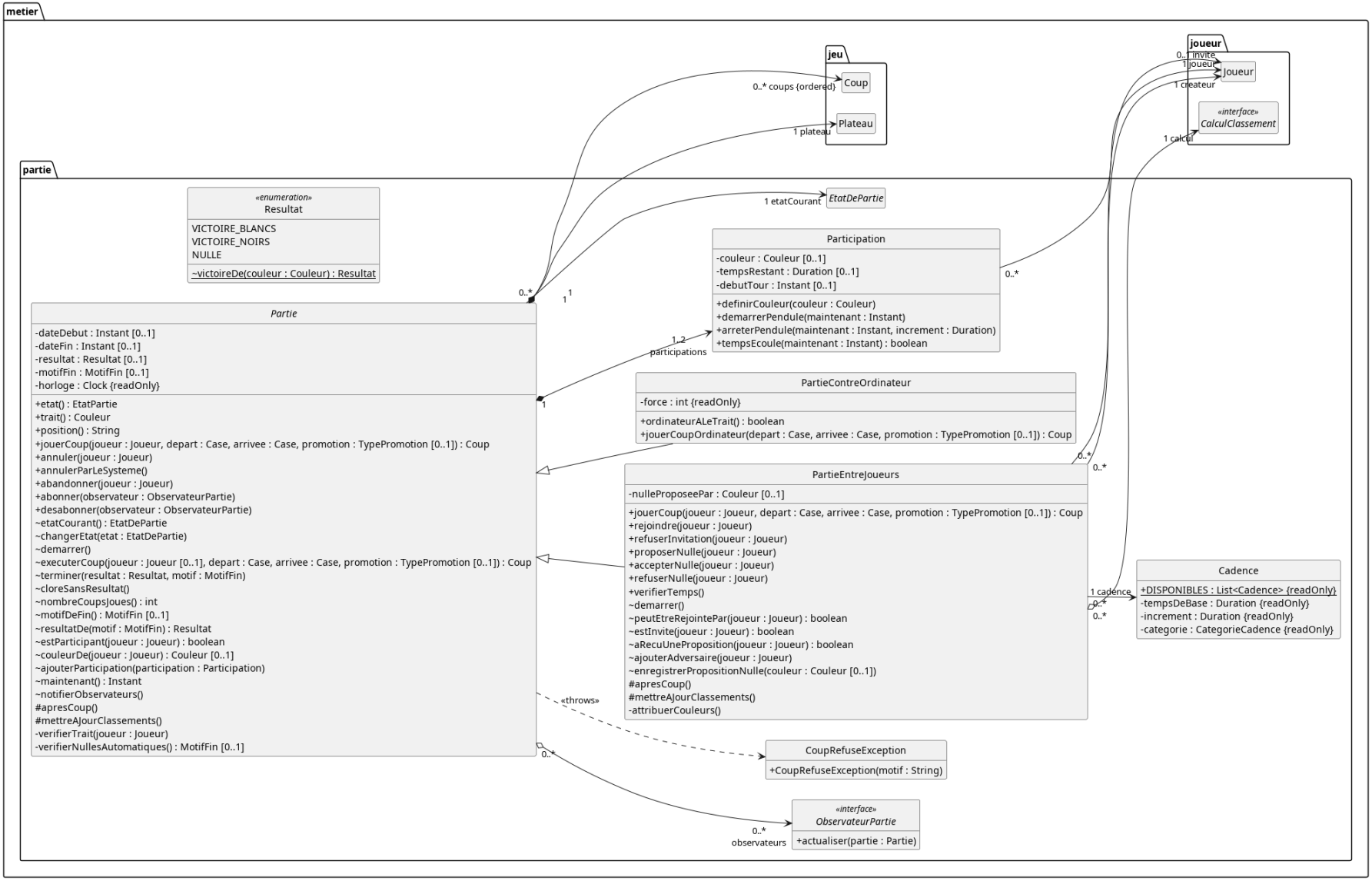


Figure 9 — Classes de conception : metier.partie

7. Architecture

Les packages organisent les couches sans cycle (section 6.2). Les composants sont des services, qui se décrivent par leurs interfaces. Le déploiement est expliqué à la section 6.8.

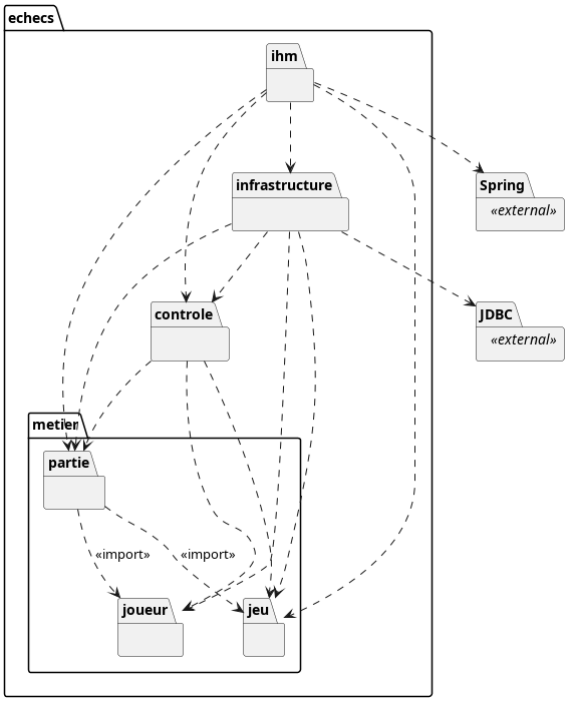


Figure 10 — Diagramme de packages

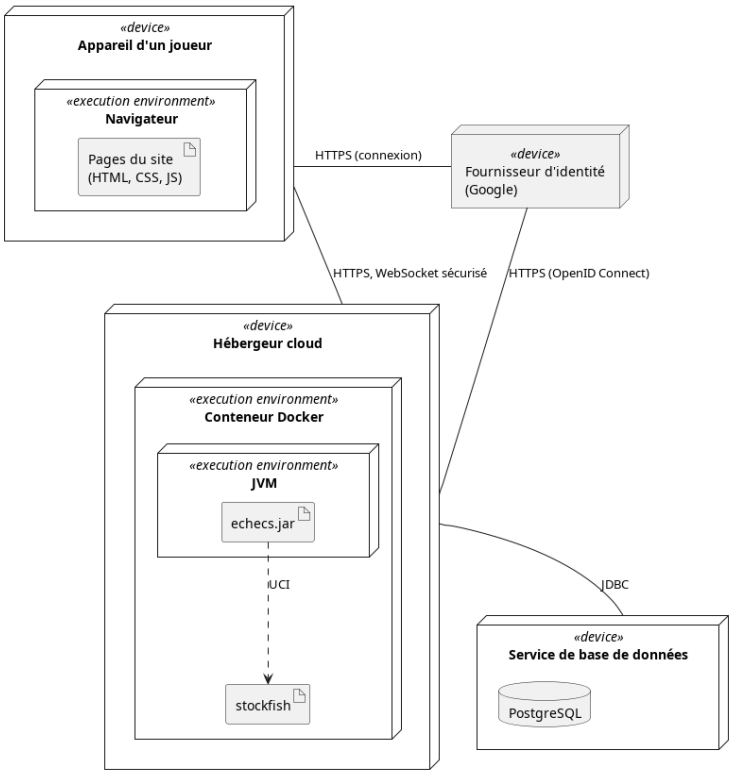


Figure 11 — Diagramme de déploiement

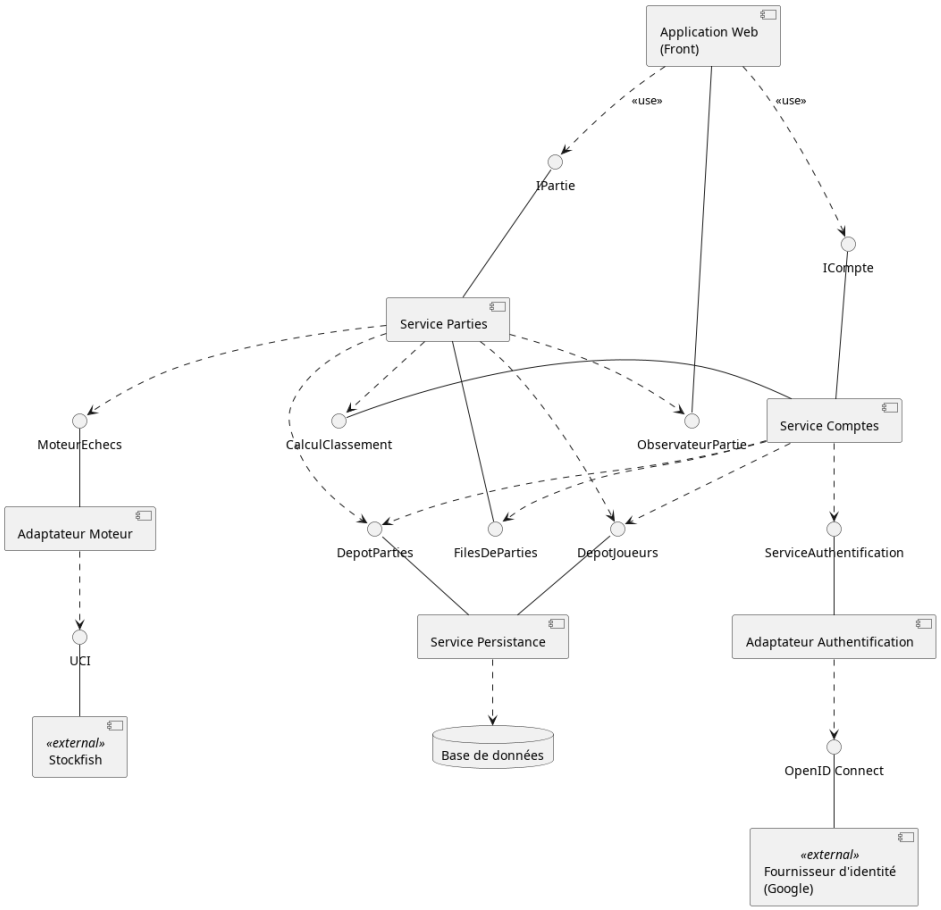


Figure 12 — Diagramme de composants

8. Traçabilité

8.1 Des cas d'utilisation aux classes et aux tests

Cas	Besoin	Classes principales	Règles	Tests
UC1 Se connecter	BF2	`ControleurCompte`, `ServiceAuthentification`, `DepotJoueurs`, `Joueur`	—	`ControleurCompteTest`
UC2 Créer un compte	BF1	`ControleurCompte`, `DepotJoueurs`, `Joueur`, `Classement`	RG12	`JoueurTest`, `ControleurCompteTest`
UC3 Créer une partie	BF3	`ControleurPartie`, `PartieEntreJoueurs`, `Cadence`, `Participation`, `EtatEnAttente`	RG8, RG11	`PartieEntreJoueursTest`, `ControleurPartieTest`
UC4 Rejoindre une partie	BF4	`ControleurPartie`, `PartieEntreJoueurs`, `EtatEnAttente`, `Participation`, `Joueur`	RG3, RG8, RG11, RG13	`PartieEntreJoueursTest`, `ControleurPartieTest`
UC5 Annuler une partie	BF5	`ControleurPartie`, `EtatEnAttente`, `EtatEnCours`, `EtatAnnulee`	RG13, RG15, RG16	`EtatsPartieTest`, `ControleurPartieTest`

Cas	Besoin	Classes principales	Règles	Tests
UC6 Jouer une partie contre un joueur	BF3, BF4	`PartieEntreJoueurs`, `Participation`, `EtatEnCours`, `EtatTerminee`, `CalculClassement`, `CalculElo`, `Classement`, `ObservateurPartie`	RG3, RG4, RG5, RG6, RG7, RG9	`PartieEntreJoueursTest`, `ParticipationTest`, `CalculEloTest`, `JoueurTest`
UC7 Jouer une partie contre l'ordinateur	BF9	`ControleurPartie`, `PartieContreOrdinateur`, `MoteurEchecs`, `AdaptateurStockfish`	RG1, RG6, RG10, RG15	`PartieContreOrdinateurTest`, `ControleurPartieTest`
UC8 Jouer un coup	BF6	`Partie`, `EtatEnCours`, `Plateau`, `Case`, `Piece`, `Coup`, `Participation`	RG1, RG2, RG3, RG4, RG7, RG9	`PlateauTest`, `PerftTest`, `CoupTest`, `PartieEntreJoueursTest`
UC9 Proposer la nulle	BF7	`PartieEntreJoueurs`, `EtatEnCours`	RG4	`PartieEntreJoueursTest`
UC10 Abandonner la partie	BF8	`Partie`, `EtatEnCours`, `EtatTerminee`	RG4, RG6	`PartieEntreJoueursTest`, `PartieContreOrdinateurTest`
UC11 Consulter ses classements	BF10	`ControleurCompte`, `Joueur`, `Classement`	RG6	`JoueurTest`, `ControleurCompteTest`
UC12 Revoir une partie terminée	BF11	`ControleurPartie`, `DepotParties`, `Partie`, `Coup`	RG7	`PartieEntreJoueursTest`
UC13 Gérer les comptes	BF12	`ControleurCompte`, `Joueur`, `DepotParties`	RG8, RG14, RG17	`JoueurTest`, `ControleurCompteTest`

Annexe A — Cahier des charges

A.1 Objectif

Un site web permettant de jouer aux échecs en ligne, avec pendule, contre un autre joueur ou contre l'ordinateur. Chaque joueur dispose d'un classement par famille de cadence et peut revoir ses parties terminées.

A.2 Acteurs

Acteur	Type	Rôle
Joueur	Principal	Crée son compte, joue des parties, consulte ses classements et ses parties
Administrateur	Principal	Gère les comptes des joueurs
Service d'authentification	Secondaire, externe	Vérifie l'identité du joueur lors de la connexion
Moteur d'échecs (Stockfish)	Secondaire, externe	Calcule les coups de l'ordinateur à la force demandée

A.3 Besoins fonctionnels

Réf.	Besoin
BF1	Créer un compte en choisissant un pseudo unique et en indiquant son adresse e-mail. Un compte est nécessaire pour accéder au site.
BF2	Se connecter : l'identité est vérifiée par le service d'authentification externe ; la connexion n'aboutit que si un compte existe pour cette identité.
BF3	Créer une partie contre un humain en choisissant la cadence et sa couleur (blanc, noir, au hasard), ouverte à tous ou en invitant un joueur précis ; la partie est « en attente ».
BF4	Consulter les parties ouvertes et en rejoindre une, ou accepter une invitation.
BF5	Annuler une partie : le créateur retire sa partie en attente, le joueur invité refuse l'invitation, ou un participant annule une partie qui vient de commencer (RG16).
BF6	Jouer un coup, y compris le choix de la pièce lors d'une promotion.
BF7	Proposer la nulle ; l'accepter ou la refuser.
BF8	Abandonner une partie.
BF9	Jouer contre l'ordinateur en choisissant sa force et sa couleur.
BF10	Consulter ses classements : un par famille de cadence (bullet, blitz, rapide).
BF11	Revoir une partie terminée coup par coup.
BF12	(Administrateur) Gérer les comptes : consulter, suspendre et réactiver un compte.

A.4 Règles de gestion

Réf.	Règle
RG1	**Coup légal.** Un coup respecte le déplacement de la pièce et ne laisse jamais son propre roi en échec. Un coup illégal est refusé.

Réf.	Règle
RG2	**Coups spéciaux.** Roque : ni le roi ni la tour n'ont bougé, aucune pièce entre eux, le roi n'est pas en échec, ne traverse pas et n'arrive pas sur une case attaquée. Prise en passant : uniquement au coup qui suit immédiatement l'avance de deux cases d'un pion adverse. Promotion : un pion atteignant la dernière rangée devient, au choix, dame, tour, fou ou cavalier.
RG3	**Tour de jeu.** Les blancs commencent. Un joueur ne joue que dans une partie à laquelle il participe, et seulement quand c'est son tour. Une partie oppose exactement deux camps ; on ne peut pas rejoindre sa propre partie.
RG4	**Fins de partie.** Mat : victoire. Pat : nulle. Abandon : défaite de celui qui abandonne. Nulle par accord des deux joueurs, par triple répétition de la position, par 50 coups sans prise ni mouvement de pion, ou par matériel insuffisant pour mater. Une proposition de nulle reste valable jusqu'à ce que l'adversaire l'accepte ou la refuse ; jouer un coup sans y répondre vaut refus.
RG5	**Pendule.** Chaque joueur dispose d'un temps qui ne décompte que pendant son tour. La cadence est choisie à la création dans une liste fixe (temps de base et incrément éventuel ajouté après chaque coup) ; chaque cadence de la liste appartient à une famille : bullet (1+0, 2+1), blitz (3+0, 3+2, 5+0) ou rapide (10+0, 10+5, 15+10), en minutes et secondes par coup. Un joueur dont le temps est écoulé perd, sauf si l'adversaire n'a pas le matériel pour mater : la partie est alors nulle.
RG6	**Classement.** Chaque joueur a un classement par famille de cadence (bullet, blitz, rapide). À la fin d'une partie entre humains, seul le classement de la famille de sa cadence est mis à jour. Les parties contre l'ordinateur ne sont pas classées. Formule retenue en J4 : Elo, avec un coefficient K = 20 et 1500 points au départ (voir le dossier de conception).
RG7	**Historique.** Une partie terminée est conservée avec tous ses coups, dans l'ordre ; elle est consultable par ses participants.
RG8	**Suspension.** Un joueur dont le compte est suspendu ne peut ni créer ni rejoindre de partie. Une partie en cours d'un joueur suspendu est considérée comme abandonnée par lui ; une partie en attente qu'il a créée est annulée.
RG9	**Nulles automatiques.** Le système déclare automatiquement la nulle par triple répétition et par 50 coups (simplification assumée : dans la règle officielle, le joueur doit la réclamer).
RG10	**Parties contre l'ordinateur.** Elles se jouent sans pendule.
RG11	**Une partie à la fois.** Un joueur a au plus une partie en attente ou en cours.
RG12	**Unicité des comptes.** Le pseudo est unique ; une adresse e-mail ne correspond qu'à un seul compte.
RG13	**Partie en attente.** Une partie en attente ne peut être annulée que par son créateur, ou par le joueur invité qui refuse l'invitation ; une partie sur invitation ne peut être rejointe que par le joueur invité ; une partie rejointe ne peut plus être annulée.
RG14	**Pas de suppression de compte.** Un compte n'est jamais supprimé, seulement suspendu, afin de conserver l'historique des parties de ses adversaires (RG7).
RG15	**Partie annulée.** Une partie annulée n'a ni résultat ni effet sur le classement. Une partie est annulée lorsque son créateur la retire ou que l'invité refuse l'invitation (partie en attente), lorsque son créateur est suspendu pendant l'attente (RG8), lorsqu'un participant l'annule en début de partie (RG16), ou lorsque le moteur d'échecs ne répond plus (partie contre l'ordinateur).
RG16	**Annulation en début de partie.** Tant que chaque camp n'a pas joué son premier coup, un participant peut annuler une partie en cours ; ensuite, seul l'abandon est possible.

Réf.	Règle
RG17	**Rôle des comptes.** Un compte a le rôle joueur ou administrateur. Seul un administrateur consulte, suspend et réactive les comptes, et nomme d'autres administrateurs. Un administrateur ne joue pas et ne peut pas être suspendu ; un joueur en partie ne peut pas être nommé. Un compte créé sur le site a toujours le rôle joueur ; le premier administrateur est créé à l'installation du site.

A.5 Choix de périmètre et justifications

Choix	Décision	Justification
Adversaire ordinateur	Moteur externe (Stockfish, protocole UCI)	Force réglable sans coder d'IA ; le moteur est un acteur secondaire, conformément au sujet. Les niveaux se réduisent à un paramètre de force ; le patron Stratégie est appliqué au calcul du classement (J4).
Authentification	Création du compte dans le système ; vérification de l'identité par un service externe (Google, avec OpenID Connect) lors de la connexion	Le compte doit exister avant le premier accès ; la sécurité des identifiants reste hors du système.
Administration	Un seul cas « Gérer les comptes », réservé au rôle administrateur (RG17)	Évite de multiplier les cas créer, modifier, supprimer (cours, §3.1.5). Un rôle sur le compte suffit : les deux acteurs partagent le compte et la connexion.
Création de compte	« Créer un compte » inclut « Se connecter »	Le joueur est connecté automatiquement dès que son compte est créé.
États de la partie	En attente, en cours, terminée, annulée ; pas d'état « abandonnée »	Un abandon est une fin de partie comme un mat : la partie est terminée, avec le motif ABANDON (RG4). Un état à part dupliquerait cette information.
Moteur d'échecs indisponible	La partie est annulée (RG15)	Un seul état « close sans résultat », quelle qu'en soit la cause ; la cause pourra être ajoutée par une énumération si un besoin apparaît.
Classement	Un classement par famille de cadence, modélisé par une association qualifiée	Comme sur chess.com : un joueur peut être fort en blitz et moyen en rapide. Le qualificatif garantit un seul classement par famille, sans contrainte supplémentaire.
Une partie à la fois	Au plus une partie en attente ou en cours (RG11)	Limiter les seules parties en cours laisserait un joueur attendre sur sa partie tout en en rejoignant une autre : si les deux démarraient, il aurait deux parties en cours.
Parties en différé	Hors périmètre : seules les familles bullet, blitz et rapide existent (RG5)	Une partie en différé dure plusieurs jours : avec RG11, le joueur ne pourrait rien jouer d'autre pendant ce temps. Si elles étaient ajoutées, RG11 ne s'appliquerait plus qu'aux parties en direct.
Annulation en début de partie	Possible tant que chaque camp n'a pas joué son premier coup (RG16)	Comme sur chess.com : une partie lancée par erreur, ou dont l'adversaire ne joue pas, ne compte pas au classement.

Choix	Décision	Justification
Annulation d'une partie en attente	Conservée	Sans elle, une partie que personne ne rejoint resterait en attente indéfiniment.
Règles d'échecs	Règles officielles complètes dans le modèle	Le modèle du domaine décrit le vrai jeu ; la priorité du code est décidée en J5.
« Rejouer une partie terminée »	Revoir la partie coup par coup	Interprétation retenue ; impose de conserver l'historique ordonné des coups.

A.6 Hors périmètre

Spectateurs (seuls les deux participants suivent une partie en cours), discussion entre joueurs, tournois, parties en différé, analyse d'une partie par le moteur, application mobile native.

Annexe B — Diagrammes complémentaires

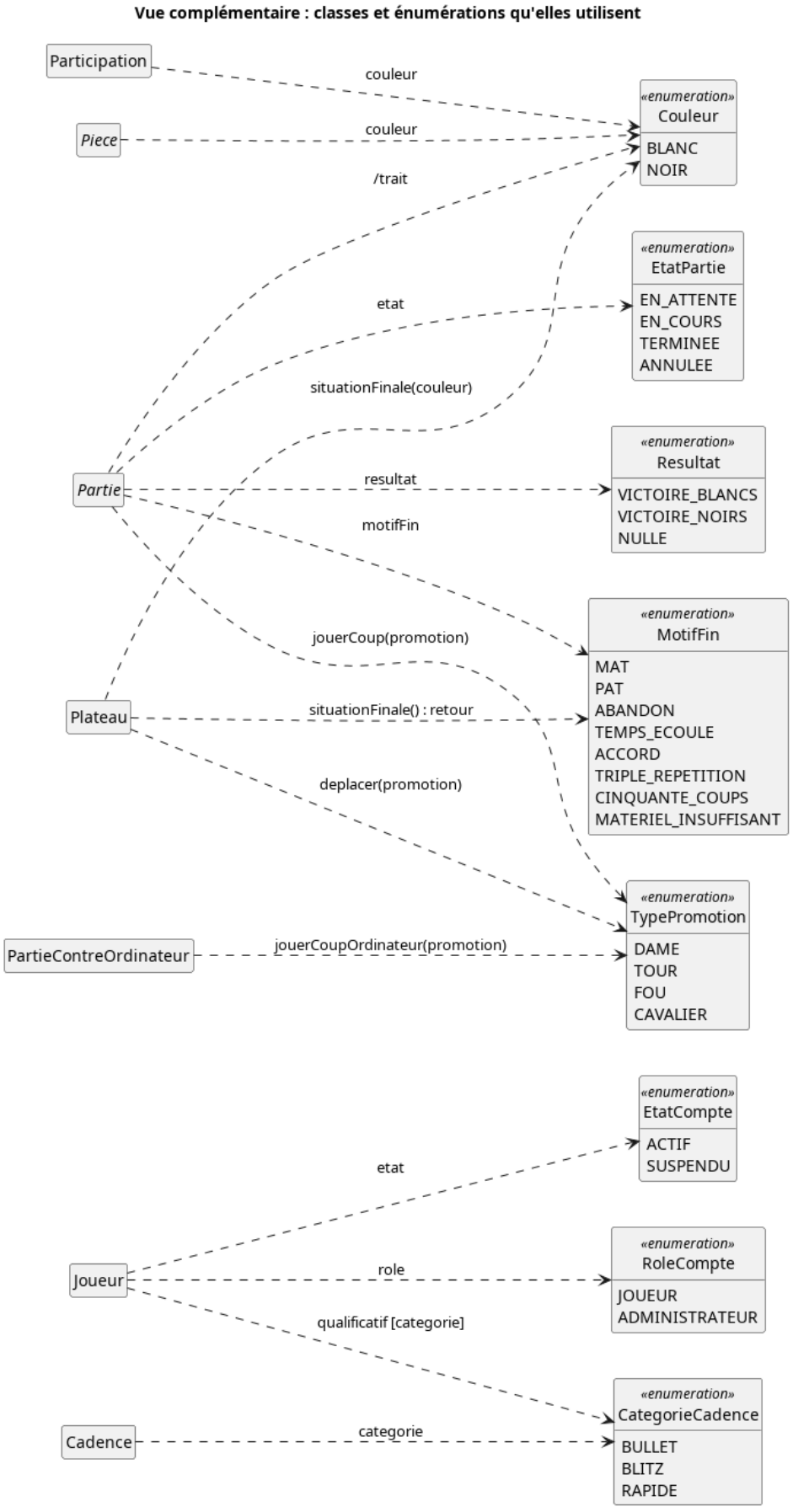


Figure 13 — Classes et énumérations qu'elles utilisent

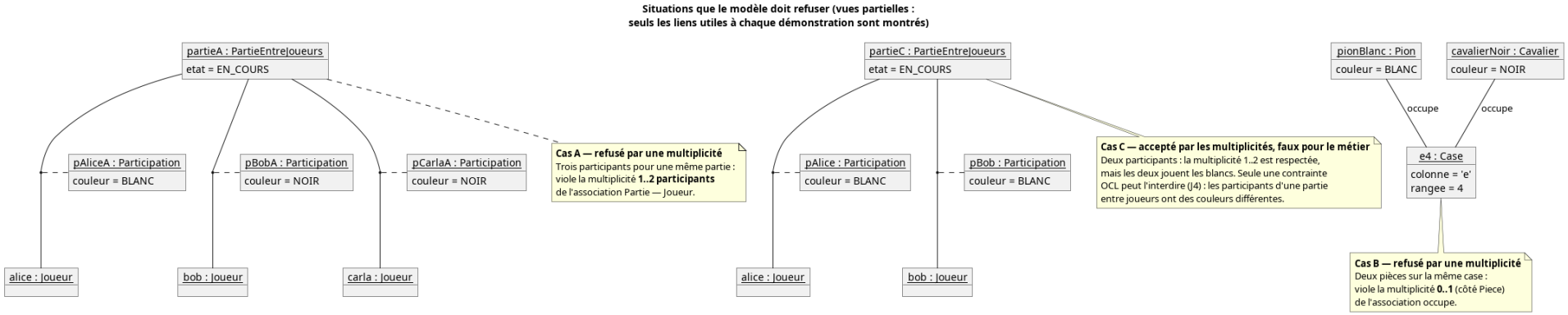


Figure 14 — Situations que le modèle doit refuser

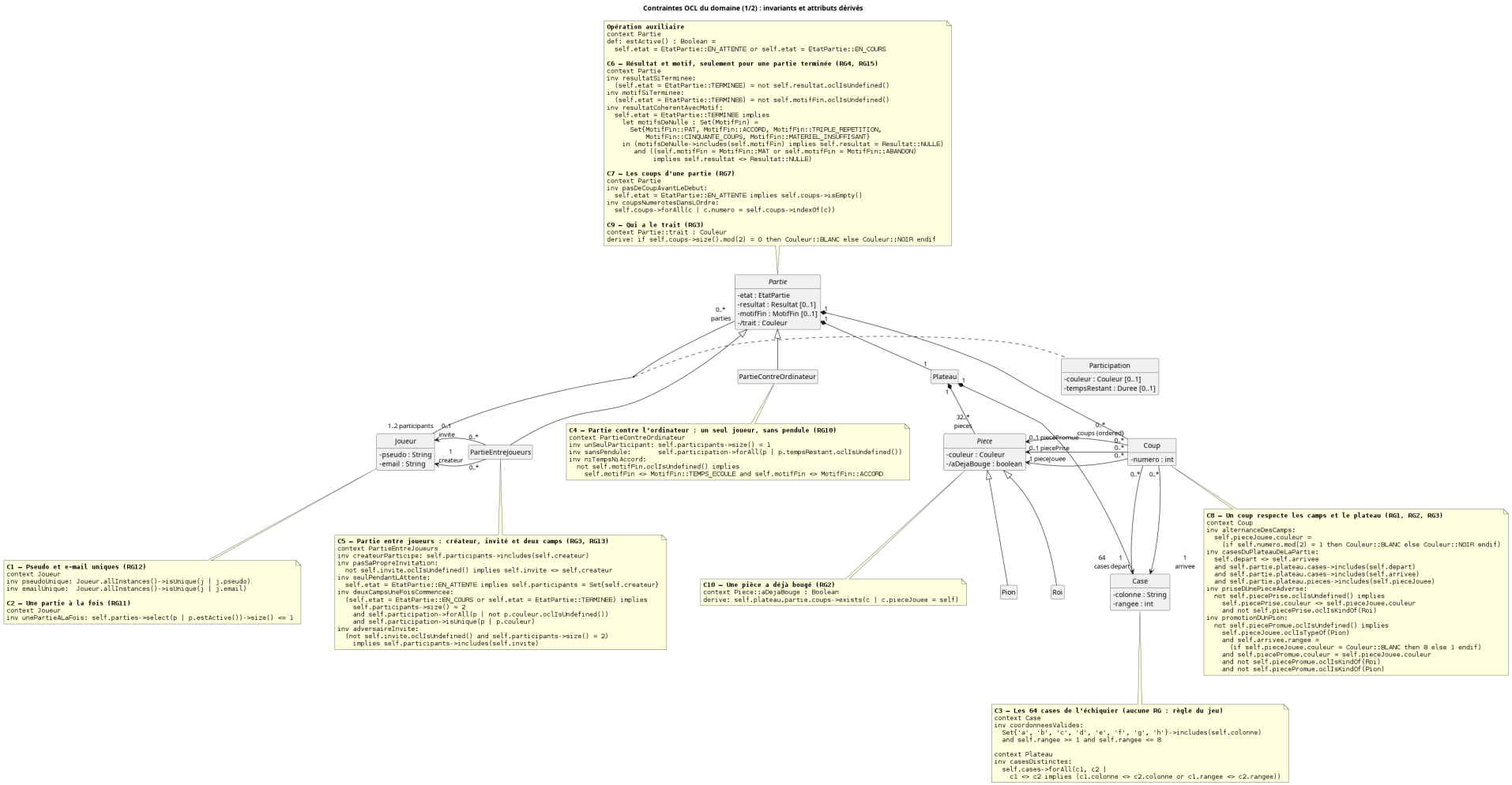


Figure 15 — Contraintes OCL (1/2) : invariants et attributs dérivés

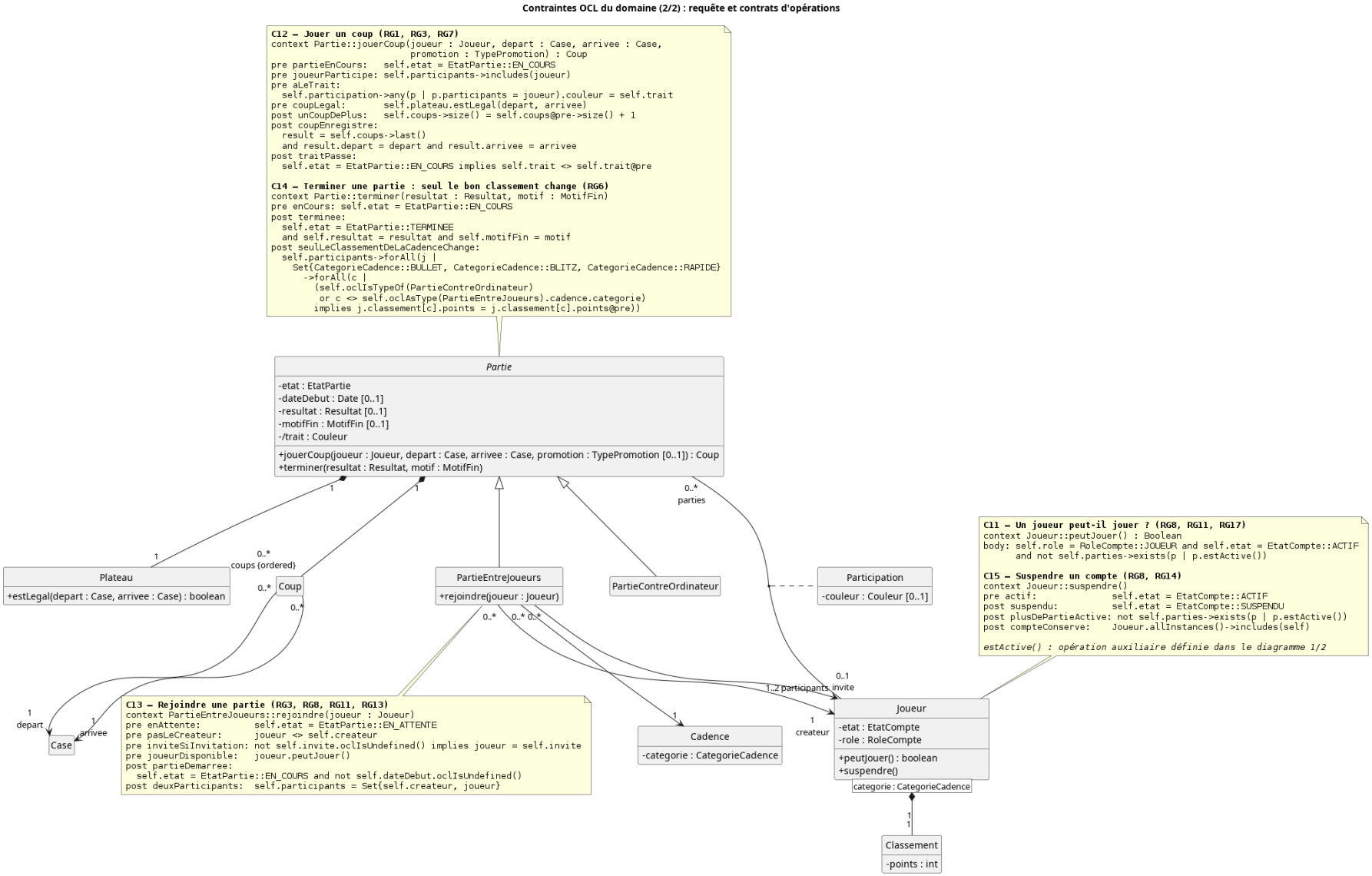


Figure 16 — Contraintes OCL (2/2) : requête et contrats d'opérations

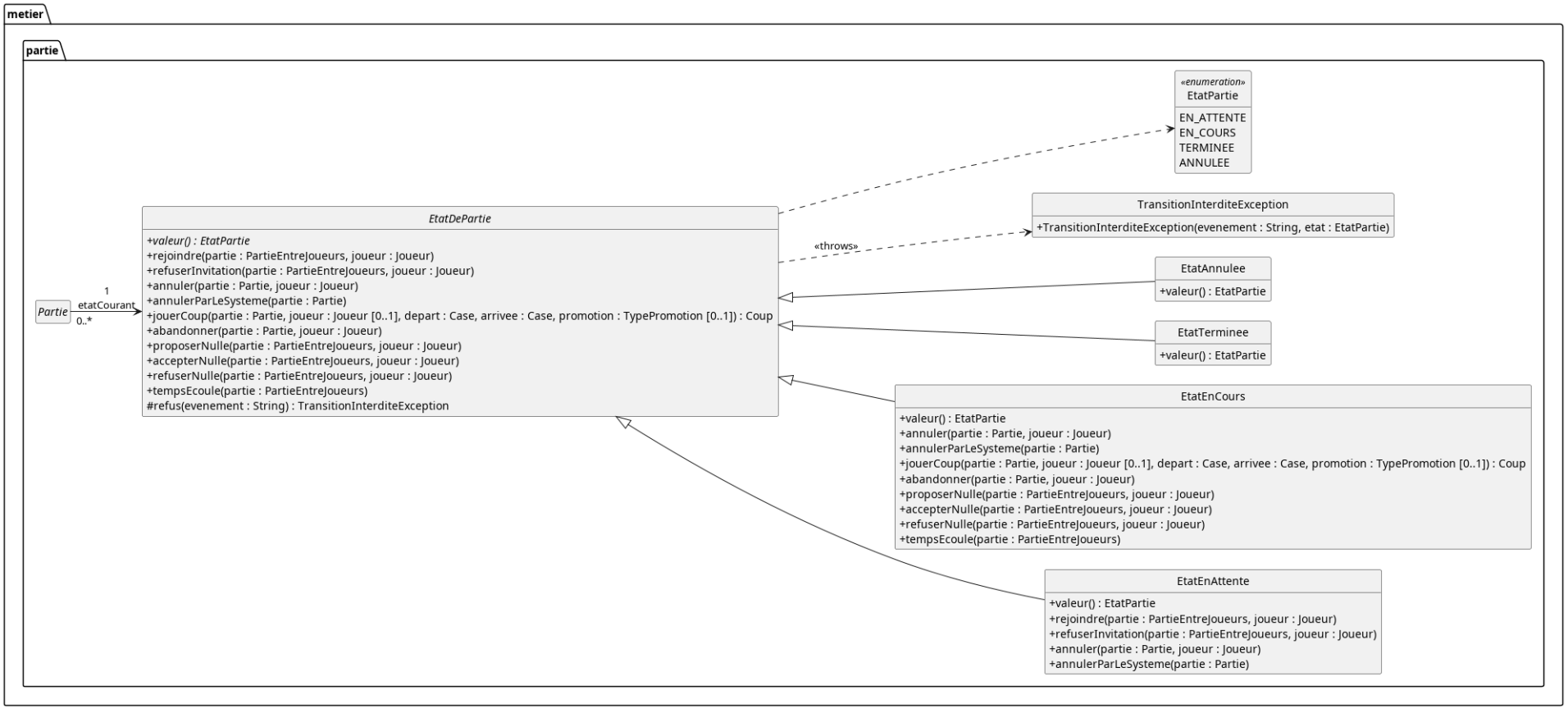


Figure 17 — Classes de conception : le patron État de metier.partie

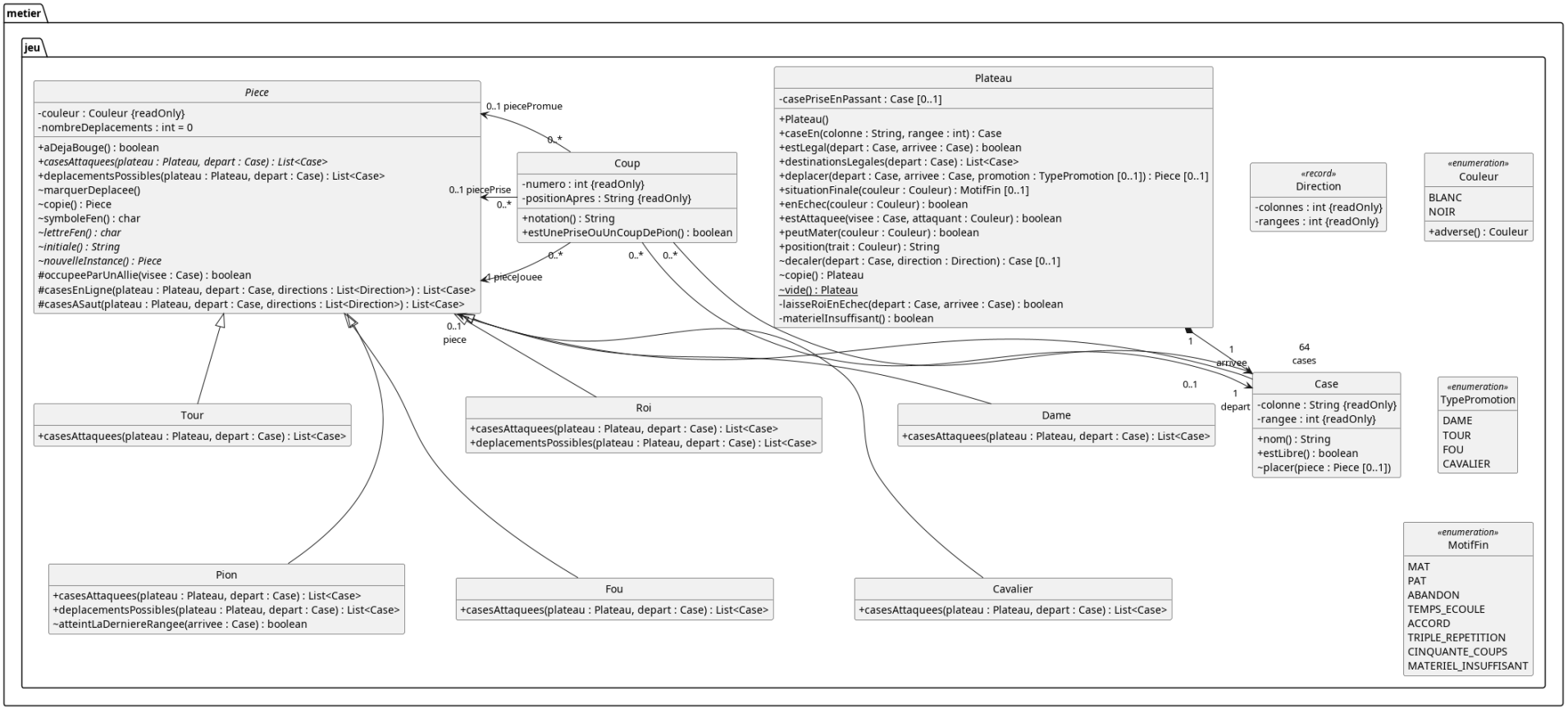


Figure 18 — Classes de conception : metier.jeu

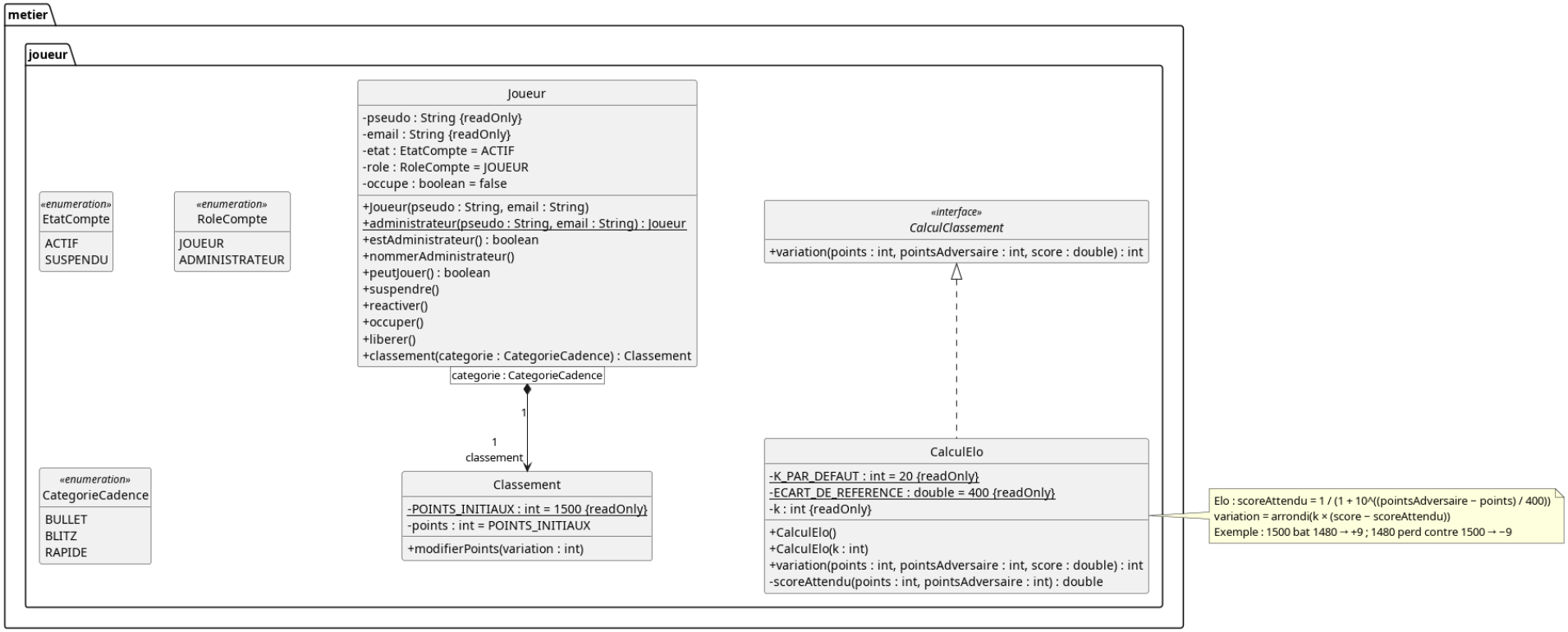


Figure 19 — Classes de conception : metier.joueur

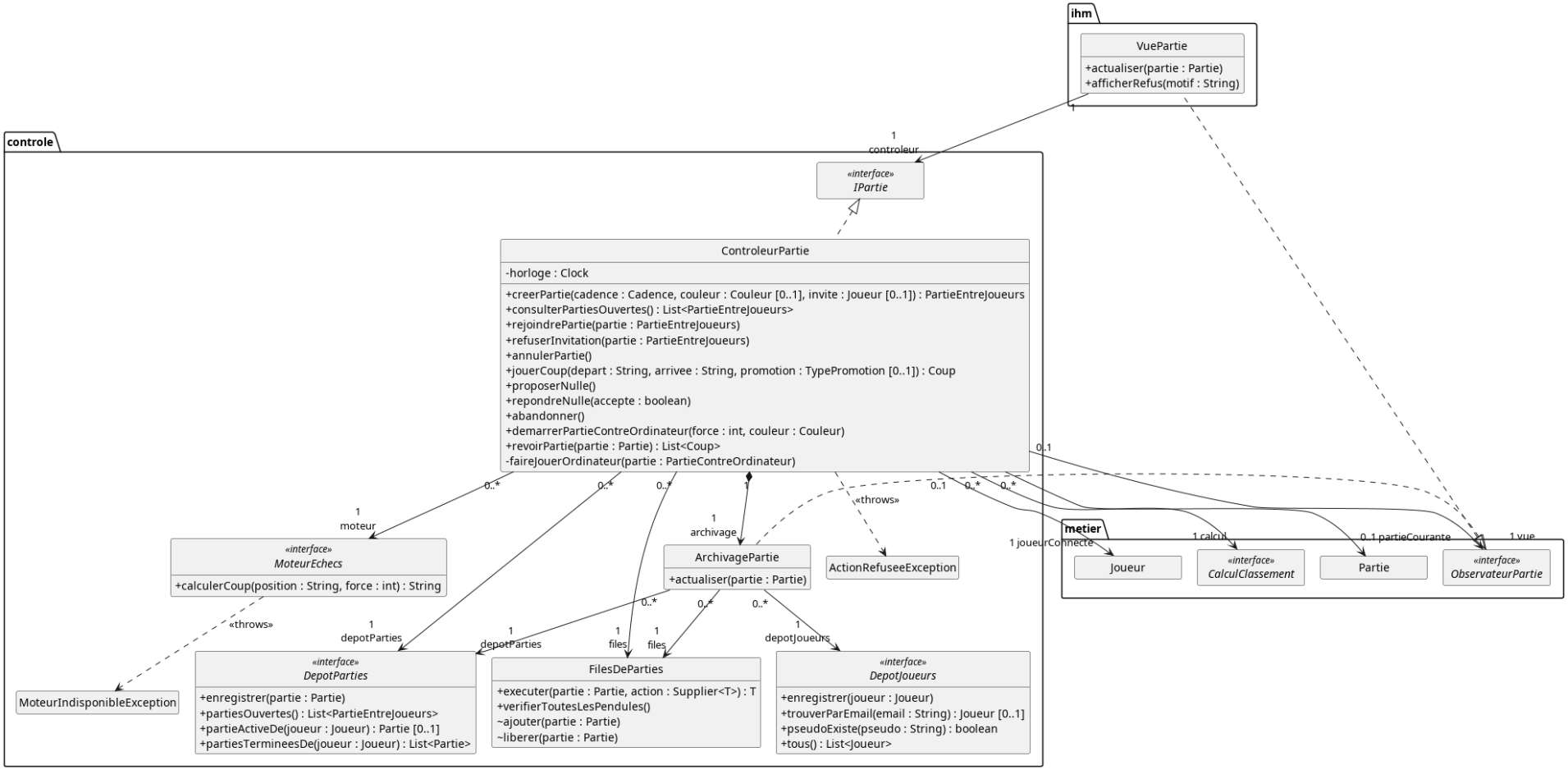


Figure 20 — Classes de conception : ihm et ControleurPartie

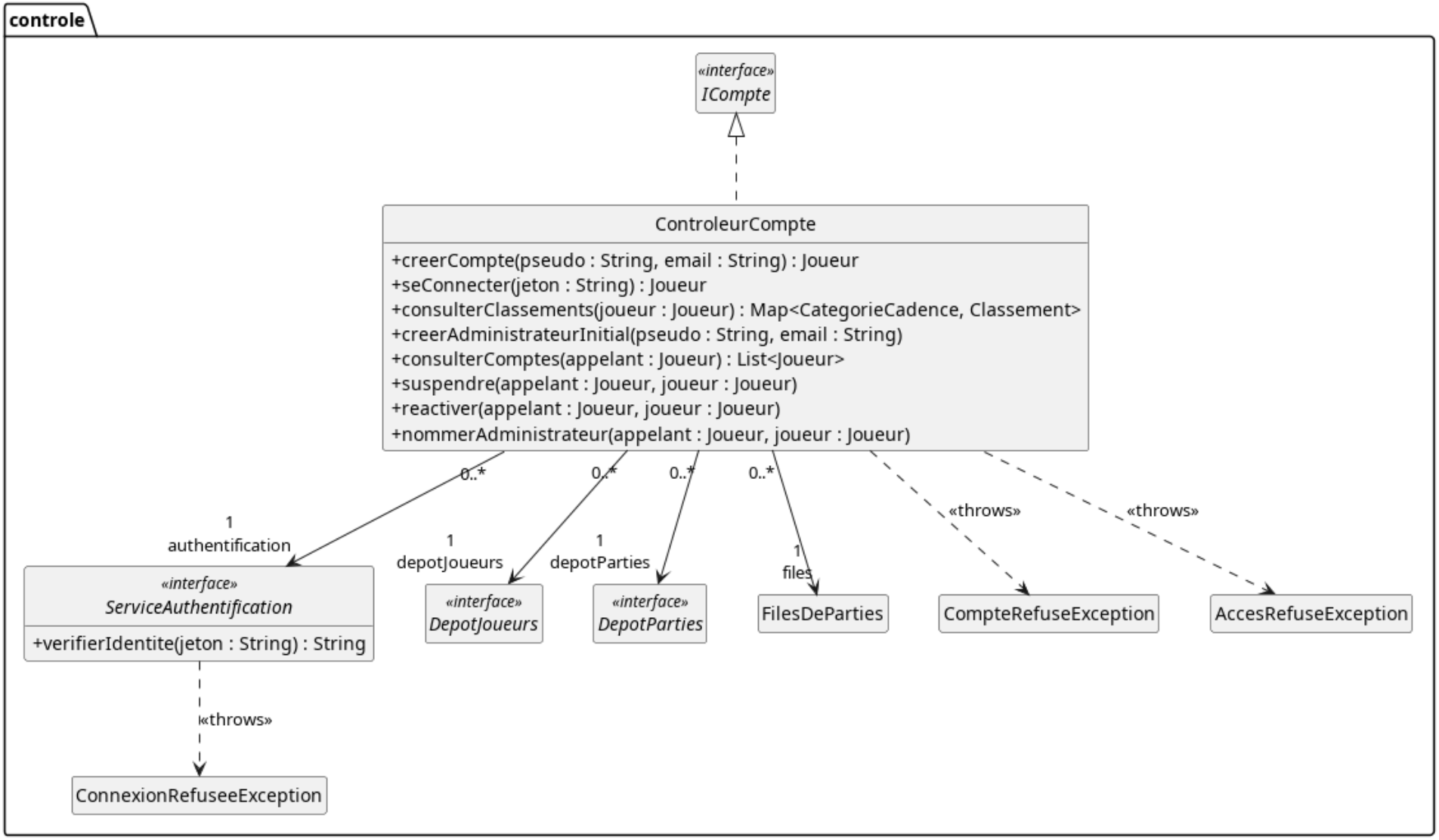


Figure 21 — Classes de conception : ControleurCompte

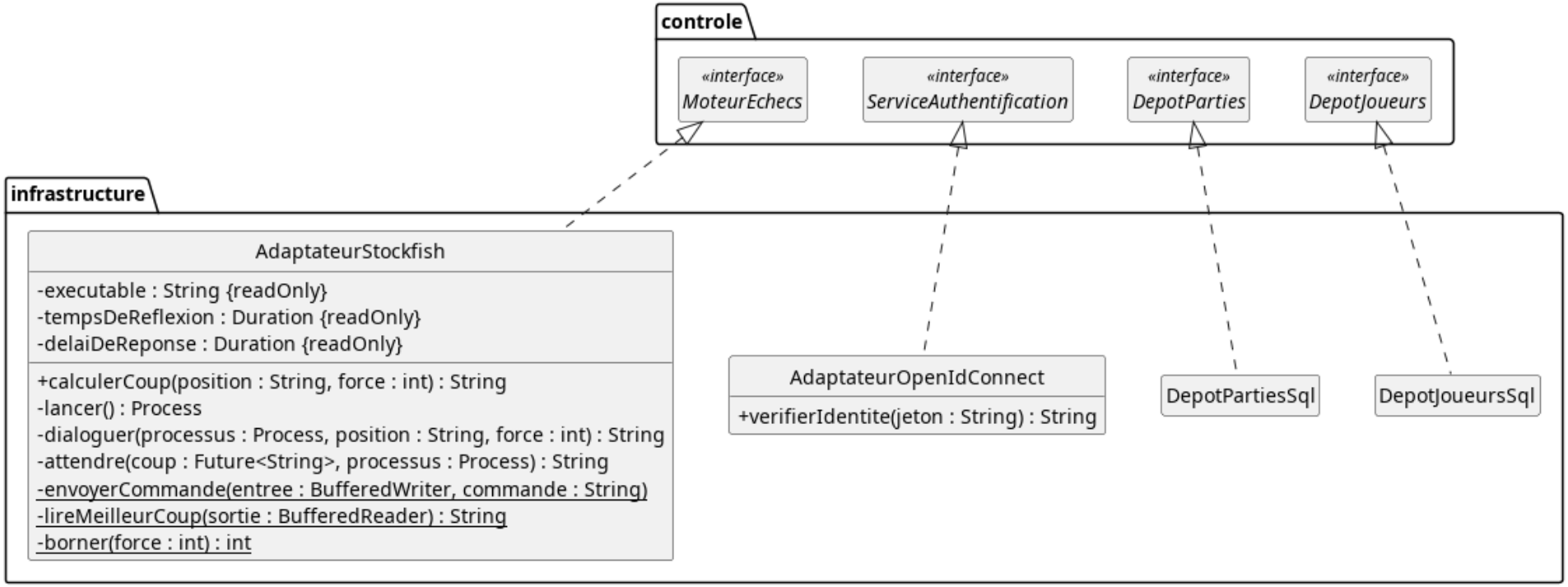


Figure 22 — Classes de conception : infrastructure

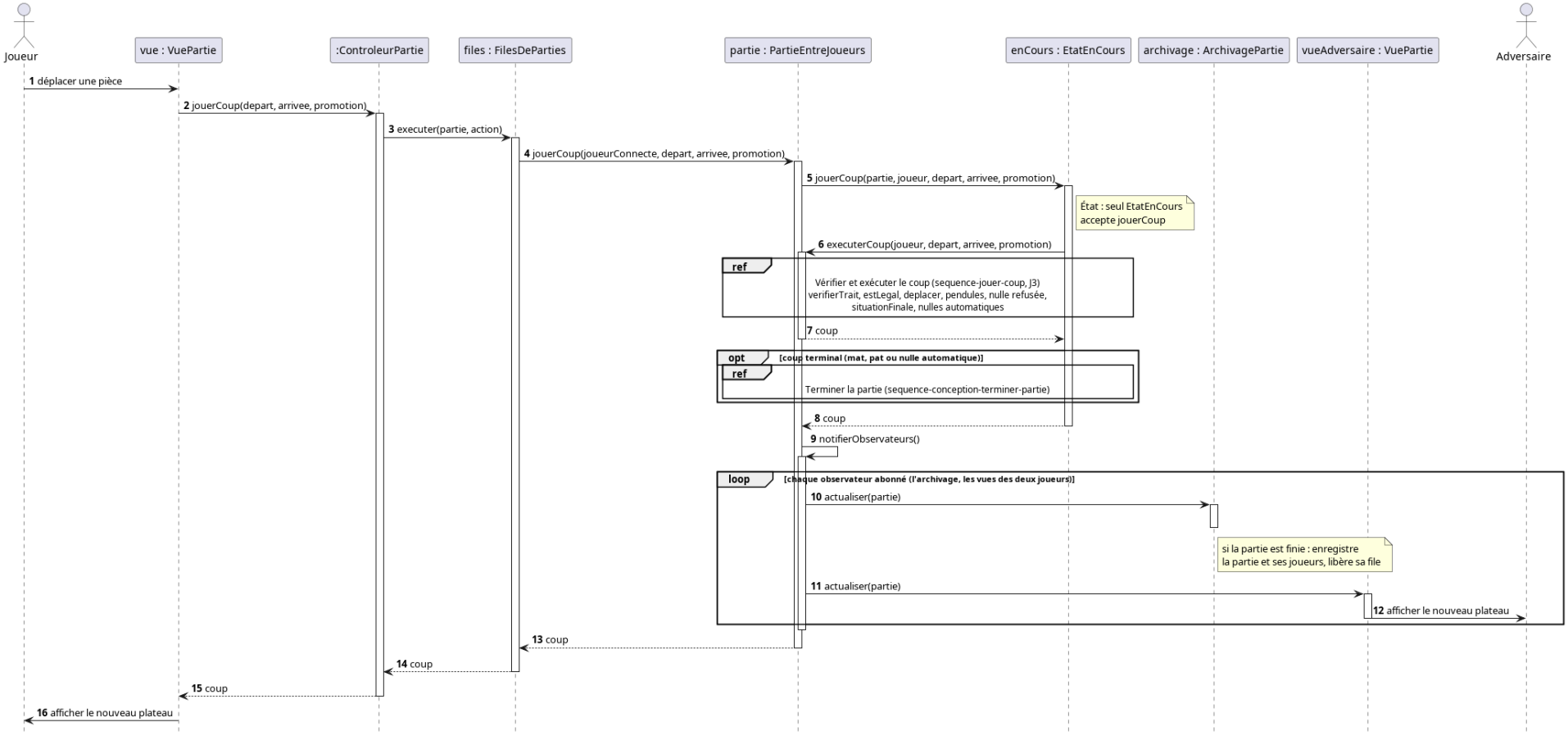


Figure 23 — Séquence de conception : jouer un coup

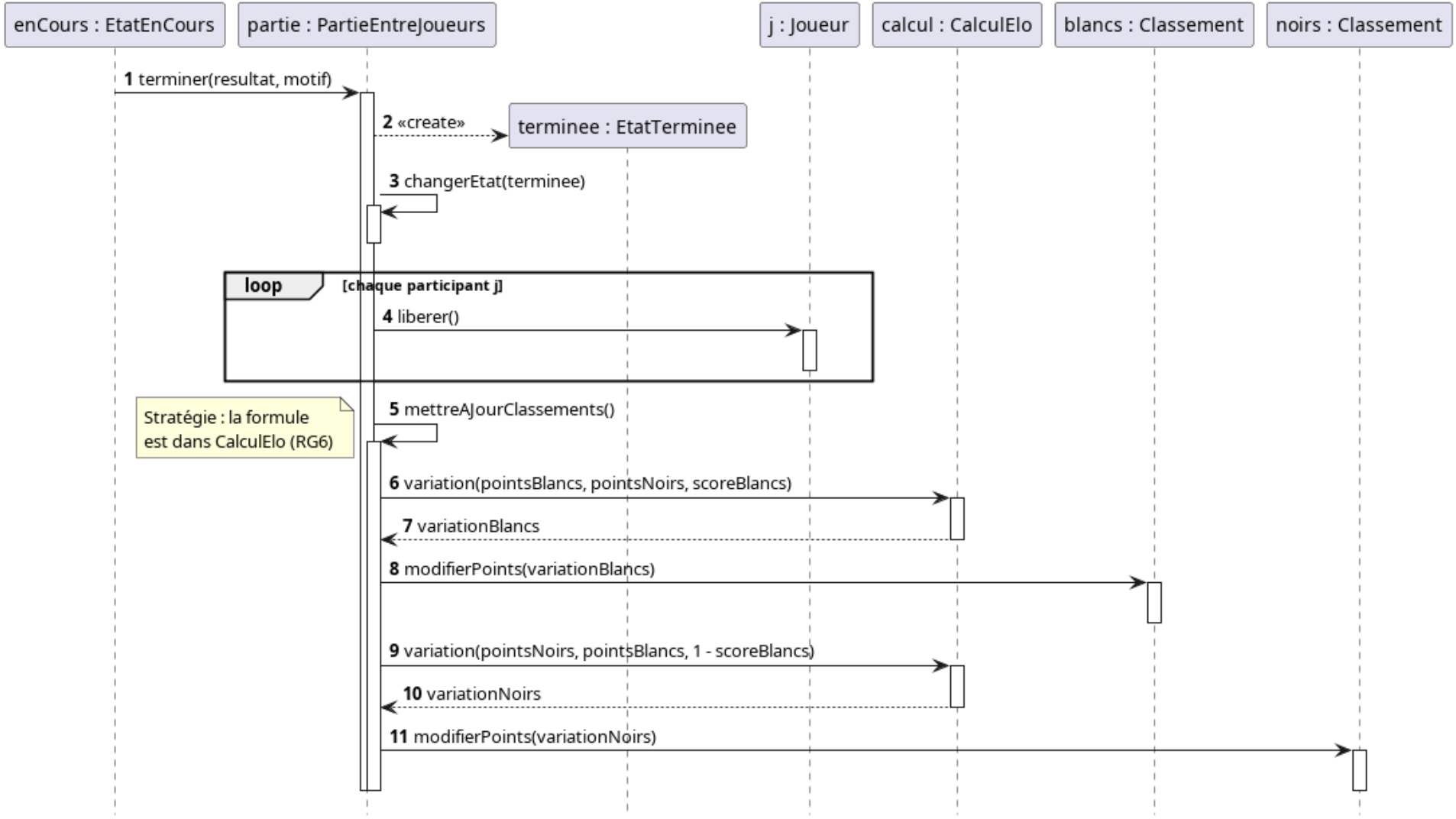


Figure 24 — Séquence de conception : terminer une partie

Annexe C — Contraintes OCL complètes

C.1 Conventions et hypothèses

- **Navigation.** On suit les noms de rôle du diagramme : `participants`, `parties`, `createur`, `invite`, `coups`, `depart`, `arrivee`, `pieceJouee`, `piecePrise`, `piecePromue`, `cases`, `pieces`. Une extrémité sans nom de rôle se désigne par le nom de sa classe en minuscules : `plateau`, `partie`, `cadence`, `classement`.
- **Classe-association.** Depuis une partie ou un joueur, `participation` donne les objets Participation. Depuis une Participation, chaque extrémité s'atteint par son nom de rôle : `p.participants` est le joueur, `p.parties` la partie.
- **Association qualifiée.** `j.classement[c]` est le classement du joueur `j` dans la catégorie `c`.
- **Types.** OCL n'a pas de type Date ni Duree : comme le cours, on suppose ces deux types munis des comparaisons usuelles. Les attributs `int` et `boolean` du diagramme sont les types OCL `Integer` et `Boolean`.
- **Préconditions et refus.** Une opération appelée depuis l'interface reçoit des demandes du réseau : elle ne fait pas confiance à l'appelant et vérifie elle-même ses préconditions (`verifierTrait`, `peutEtreRejointePar`, `peutJouer`). Une précondition non remplie produit le refus montré dans les diagrammes de séquence ; la postcondition décrit le cas où l'opération aboutit.

C.2 Opération auxiliaire

Trois contraintes ont besoin de savoir si une partie est « active », c'est-à-dire en attente ou en cours. On le définit une fois.

```
context Partie
def: estActive() : Boolean =
  self.etat = EtatPartie::EN_ATTENTE or self.etat = EtatPartie::EN_COURS
```

C.3 Joueurs

C1 — Pseudo et e-mail uniques (RG12)

Deux joueurs ne partagent ni un pseudo ni une adresse e-mail.

```
context Joueur
inv pseudoUnique: Joueur.allInstances()->isUnique(j | j.pseudo)
inv emailUnique:  Joueur.allInstances()->isUnique(j | j.email)
```

L'unicité porte sur l'ensemble des joueurs : aucune multiplicité ne peut l'exprimer, et `{unique}` ne concerne que les valeurs d'un attribut multivalué.

C2 — Une partie à la fois (RG11)

Un joueur participe à au plus une partie en attente ou en cours.

```
context Joueur
inv unePartieALaFois: self.parties->select(p | p.estActive())->size() <= 1
```

La multiplicité `0..*` côté `parties` est nécessaire pour l'historique (RG7) : seule une contrainte peut limiter les parties actives sans limiter les parties terminées.

C.4 Plateau

C3 — Les 64 cases de l'échiquier (aucune RG : règle du jeu)

Chaque case a une colonne de a à h et une rangée de 1 à 8, et deux cases d'un même plateau n'ont pas les mêmes coordonnées.

```
context Case
inv coordonneesValides:
  Set{'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'}->includes(self.colonne)
  and self.rangee >= 1 and self.rangee <= 8

context Plateau
inv casesDistinctes:
  self.cases->forAll(c1, c2 |
    c1 <> c2 implies (c1.colonne <> c2.colonne or c1.rangee <> c2.rangee))
```

La multiplicité donne 64 cases ; ces deux invariants garantissent que ce sont les 64 cases de l'échiquier, chacune une seule fois.

C.5 Parties

C4 — Partie contre l'ordinateur : un seul joueur, sans pendule (RG10)

```
context PartieContreOrdinateur
inv unSeulParticipant: self.participants->size() = 1
inv sansPendule:       self.participation->forAll(p | p.tempsRestant.oclIsUndefined())
inv niTempsNiAccord:
  not self.motifFin.oclIsUndefined() implies
    self.motifFin <> MotifFin::TEMPS_ECOULE and self.motifFin <> MotifFin::ACCORD
```

L'association `1..2 participants` est définie sur Partie, pour les deux sortes de parties : la sous-classe la restreint à 1. Sans pendule, la partie ne peut pas se perdre au temps ; sans adversaire humain, personne ne propose la nulle.

C5 — Partie entre joueurs : créateur, invité et deux camps (RG3, RG13)

Le créateur participe toujours à sa partie ; il ne peut pas s'inviter lui-même ; tant que la partie attend, il est seul ; une fois commencée, elle oppose deux joueurs de couleurs différentes, dont l'invité s'il y en a un.

```
context PartieEntreJoueurs
inv createurParticipe: self.participants->includes(self.createur)
inv pasSaPropreInvitation:
  not self.invite.oclIsUndefined() implies self.invite <> self.createur
inv seulPendantLAttente:
  self.etat = EtatPartie::EN_ATTENTE implies self.participants = Set{self.createur}
inv deuxCampsUneFoisCommencee:
  (self.etat = EtatPartie::EN_COURS or self.etat = EtatPartie::TERMINEE) implies
    self.participants->size() = 2
    and self.participation->forAll(p | not p.couleur.oclIsUndefined())
    and self.participation->isUnique(p | p.couleur)
inv adversaireInvite:
  (not self.invite.oclIsUndefined() and self.participants->size() = 2)
    implies self.participants->includes(self.invite)
```

Le diagramme d'objets « impossible » (cas C) montre une partie que les multiplicités acceptent, avec deux participants blancs : `deuxCampsUneFoisCommencee` la refuse.

C6 — Résultat et motif, seulement pour une partie terminée (RG4, RG15)

Une partie a un résultat et un motif de fin si et seulement si elle est terminée ; une partie annulée n'en a donc pas. Le résultat doit être cohérent avec le motif.

```
context Partie
inv resultatSiTerminee:
  (self.etat = EtatPartie::TERMINEE) = not self.resultat.oclIsUndefined()
inv motifSiTerminee:
```

```
  (self.etat = EtatPartie::TERMINEE) = not self.motifFin.oclIsUndefined()
inv resultatCoherentAvecMotif:
  self.etat = EtatPartie::TERMINEE implies
    let motifsDeNulle : Set(MotifFin) =
      Set{MotifFin::PAT, MotifFin::ACCORD, MotifFin::TRIPLE_REPETITION,
          MotifFin::CINQUANTE_COUPS, MotifFin::MATERIEL_INSUFFISANT}
    in (motifsDeNulle->includes(self.motifFin) implies self.resultat = Resultat::NULLE)
       and ((self.motifFin = MotifFin::MAT or self.motifFin = MotifFin::ABANDON)
            implies self.resultat <> Resultat::NULLE)
```

Les multiplicités `[0..1]` disent seulement que les valeurs sont facultatives, pas quand elles doivent être présentes. TEMPS_ECOULE n'est pas contraint : il donne une défaite, ou une nulle si l'adversaire ne peut pas mater (RG5).

C7 — Les coups d'une partie (RG7)

Une partie en attente n'a aucun coup ; les coups sont numérotés 1, 2, 3… dans l'ordre où ils ont été joués.

```
context Partie
inv pasDeCoupAvantLeDebut:
  self.etat = EtatPartie::EN_ATTENTE implies self.coups->isEmpty()
inv coupsNumerotesDansLOrdre:
  self.coups->forAll(c | c.numero = self.coups->indexOf(c))
```

{ordered} dit que les coups sont rangés ; l'invariant relie ce rang à l'attribut `numero`, utilisé pour revoir une partie coup par coup (BF11).

C.6 Coups

C8 — Un coup respecte les camps et le plateau (RG1, RG2, RG3)

Les blancs jouent les coups impairs et les noirs les coups pairs ; une pièce part d'une case et arrive sur une autre, du plateau de sa partie ; on ne prend que des pièces adverses et jamais le roi ; seul un pion arrivé sur la dernière rangée est promu, en une pièce de sa couleur qui n'est ni un roi ni un pion.

```
context Coup
inv alternanceDesCamps:
  self.pieceJouee.couleur =
    (if self.numero.mod(2) = 1 then Couleur::BLANC else Couleur::NOIR endif)
inv casesDuPlateauDeLaPartie:
  self.depart <> self.arrivee
  and self.partie.plateau.cases->includes(self.depart)
  and self.partie.plateau.cases->includes(self.arrivee)
  and self.partie.plateau.pieces->includes(self.pieceJouee)
inv priseDUnePieceAdverse:
  not self.piecePrise.oclIsUndefined() implies
    self.piecePrise.couleur <> self.pieceJouee.couleur
    and not self.piecePrise.oclIsKindOf(Roi)
inv promotionDUnPion:
  not self.piecePromue.oclIsUndefined() implies
    self.pieceJouee.oclIsTypeOf(Pion)
    and self.arrivee.rangee =
      (if self.pieceJouee.couleur = Couleur::BLANC then 8 else 1 endif)
    and self.piecePromue.couleur = self.pieceJouee.couleur
    and not self.piecePromue.oclIsKindOf(Roi)
    and not self.piecePromue.oclIsKindOf(Pion)
```

Les associations disent qu'un coup a une case de départ, une case d'arrivée et des pièces ; elles ne disent pas lesquelles sont permises. La légalité complète d'un déplacement (trajectoire, échec au roi) reste calculée par `Plateau::estLegal`.

C.7 Attributs dérivés et requêtes

C9 — Qui a le trait (RG3)

Les blancs commencent : le trait est aux blancs après un nombre pair de coups.

```
context Partie::trait : Couleur
derive: if self.coups->size().mod(2) = 0 then Couleur::BLANC else Couleur::NOIR endif
```

C10 — Une pièce a déjà bougé (RG2)

Une pièce a déjà bougé si elle a été jouée dans un coup de sa partie. Le roque et l'avance de deux cases du pion en dépendent.

```
context Piece::aDejaBouge : Boolean
derive: self.plateau.partie.coups->exists(c | c.pieceJouee = self)
```

C11 — Un joueur peut-il jouer ? (RG8, RG11, RG17)

```
context Joueur::peutJouer() : Boolean
body: self.role = RoleCompte::JOUEUR and self.etat = EtatCompte::ACTIF
      and not self.parties->exists(p | p.estActive())
```

Les séquences « rejoindre une partie » et « contre l'ordinateur » appellent `peutJouer()` : cette définition fixe exactement ce qu'elle vérifie.

C.8 Contrats d'opérations

C12 — Jouer un coup (RG1, RG3, RG7)

```
context Partie::jouerCoup(joueur : Joueur, depart : Case, arrivee : Case,
                          promotion : TypePromotion) : Coup
pre partieEnCours:   self.etat = EtatPartie::EN_COURS
pre joueurParticipe: self.participants->includes(joueur)
pre aLeTrait:
  self.participation->any(p | p.participants = joueur).couleur = self.trait
pre coupLegal:       self.plateau.estLegal(depart, arrivee)
post unCoupDePlus:   self.coups->size() = self.coups@pre->size() + 1
post coupEnregistre:
  result = self.coups->last()
  and result.depart = depart and result.arrivee = arrivee
post traitPasse:
  self.etat = EtatPartie::EN_COURS implies self.trait <> self.trait@pre
```

C13 — Rejoindre une partie (RG3, RG8, RG11, RG13)

```
context PartieEntreJoueurs::rejoindre(joueur : Joueur)
pre enAttente:          self.etat = EtatPartie::EN_ATTENTE
pre pasLeCreateur:      joueur <> self.createur
pre inviteSiInvitation: not self.invite.oclIsUndefined() implies joueur = self.invite
pre joueurDisponible:   joueur.peutJouer()
post partieDemarree:
  self.etat = EtatPartie::EN_COURS and not self.dateDebut.oclIsUndefined()
post deuxParticipants:  self.participants = Set{self.createur, joueur}
```

C14 — Terminer une partie : seul le bon classement change (RG6)

À la fin d'une partie entre joueurs, seul le classement de la catégorie de sa cadence peut changer ; à la fin d'une partie contre l'ordinateur, aucun ne change.

```
context Partie::terminer(resultat : Resultat, motif : MotifFin)
pre enCours: self.etat = EtatPartie::EN_COURS
post terminee:
  self.etat = EtatPartie::TERMINEE
  and self.resultat = resultat and self.motifFin = motif
post seulLeClassementDeLaCadenceChange:
```

```
self.participants->forAll(j |
  Set{CategorieCadence::BULLET, CategorieCadence::BLITZ, CategorieCadence::RAPIDE}
    ->forAll(c |
      (self.oclIsTypeOf(PartieContreOrdinateur)
       or c <> self.oclAsType(PartieEntreJoueurs).cadence.categorie)
      implies j.classement[c].points = j.classement[c].points@pre))
```

La formule de calcul des points (patron Stratégie) n'est pas fixée ici : le contrat dit seulement quels classements elle a le droit de modifier.

C15 — Suspendre un compte (RG8, RG14)

Après la suspension, le joueur n'a plus de partie active, et son compte existe toujours.

```
context Joueur::suspendre()
pre actif:              self.etat = EtatCompte::ACTIF
post suspendu:          self.etat = EtatCompte::SUSPENDU
post plusDePartieActive: not self.parties->exists(p | p.estActive())
post compteConserve:    Joueur.allInstances()->includes(self)
```

`plusDePartieActive` résume RG8 : sa partie en attente est annulée, sa partie en cours est terminée par son abandon. `compteConserve` exprime RG14 : suspendre n'est pas supprimer.

C.9 Récapitulatif

N°	Sorte	Contexte	Règles
—	def	Partie::estActive()	RG11
C1	inv (2)	Joueur	RG12
C2	inv	Joueur	RG11
C3	inv (2)	Case, Plateau	Règle du jeu
C4	inv (3)	PartieContreOrdinateur	RG5, RG10
C5	inv (5)	PartieEntreJoueurs	RG3, RG13
C6	inv (3)	Partie	RG4, RG5, RG15
C7	inv (2)	Partie	RG7
C8	inv (4)	Coup	RG1, RG2, RG3
C9	derive	Partie::trait	RG3
C10	derive	Piece::aDejaBouge	RG2
C11	body	Joueur::peutJouer()	RG8, RG11, RG17
C12	pre (4), post (3)	Partie::jouerCoup()	RG1, RG3, RG7
C13	pre (4), post (2)	PartieEntreJoueurs::rejoindre()	RG3, RG8, RG11, RG13
C14	pre, post (2)	Partie::terminer()	RG6
C15	pre, post (3)	Joueur::suspendre()	RG8, RG14

Soit 15 contraintes pour 8 exigées. Elles utilisent six des sept mots-clés du cours : `def`, `inv`, `derive`, `body`, `pre` et `post` ; seul `init` est inutile, les valeurs initiales figurant déjà dans le diagramme (`etat = EN_ATTENTE`, `etat = ACTIF`).